

Lecture 5

Fundamentals of Deep Learning & Neural Networks

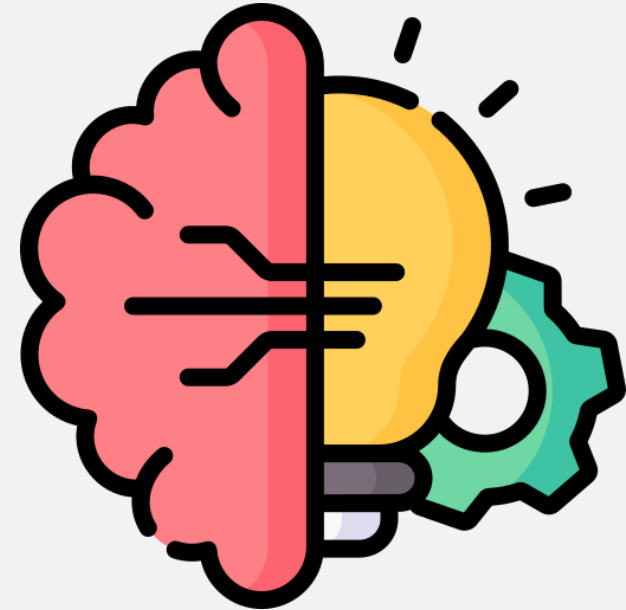
Course overview

- **Learning Objectives**
 - Introduce and familiarise you with the Basics of Deep Learning
 - Learning how multi- perceptron's works, activation functions, and propagation
 - Understanding about the problems in each NN followed by CNN

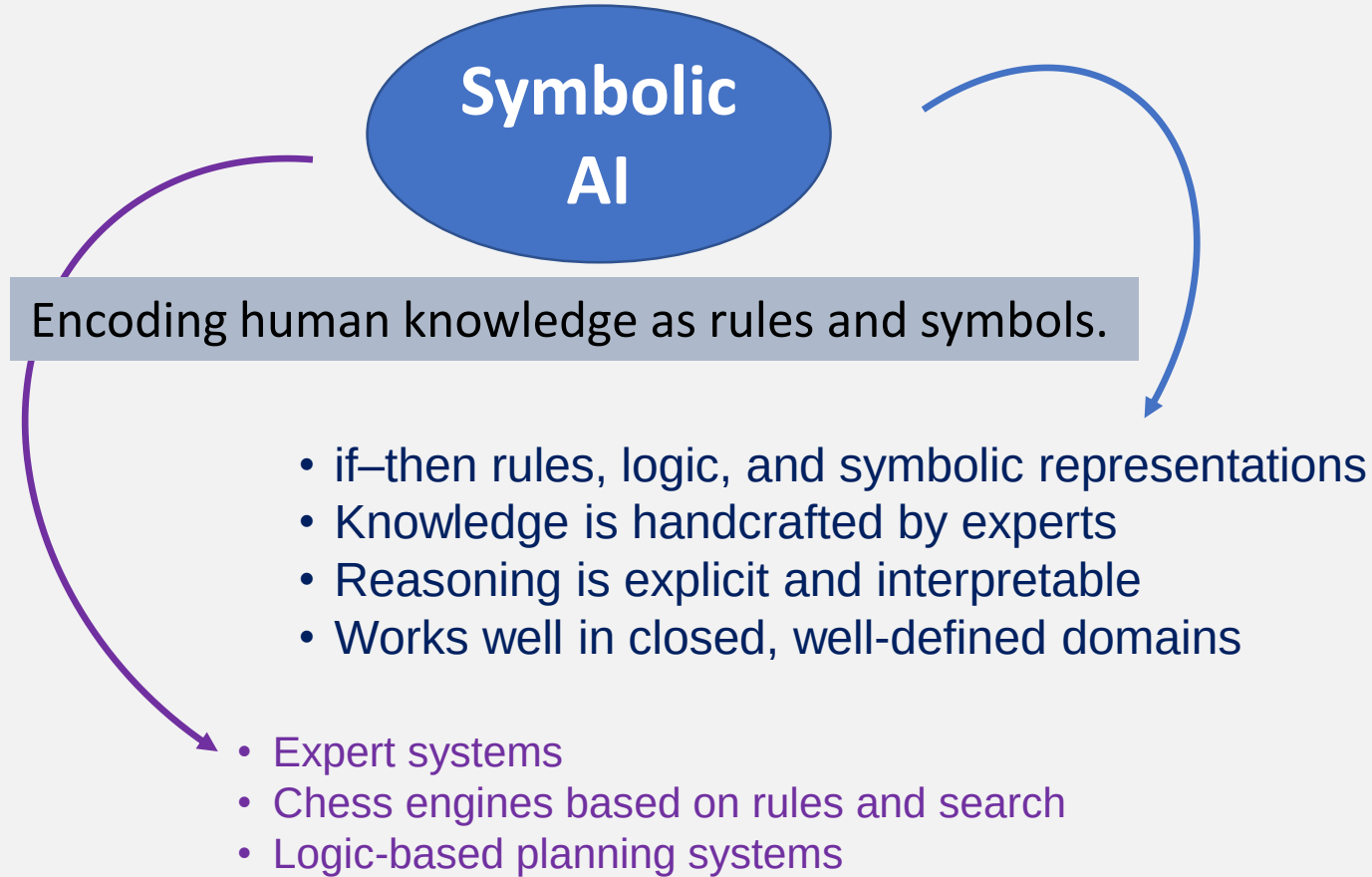


Table's Contents

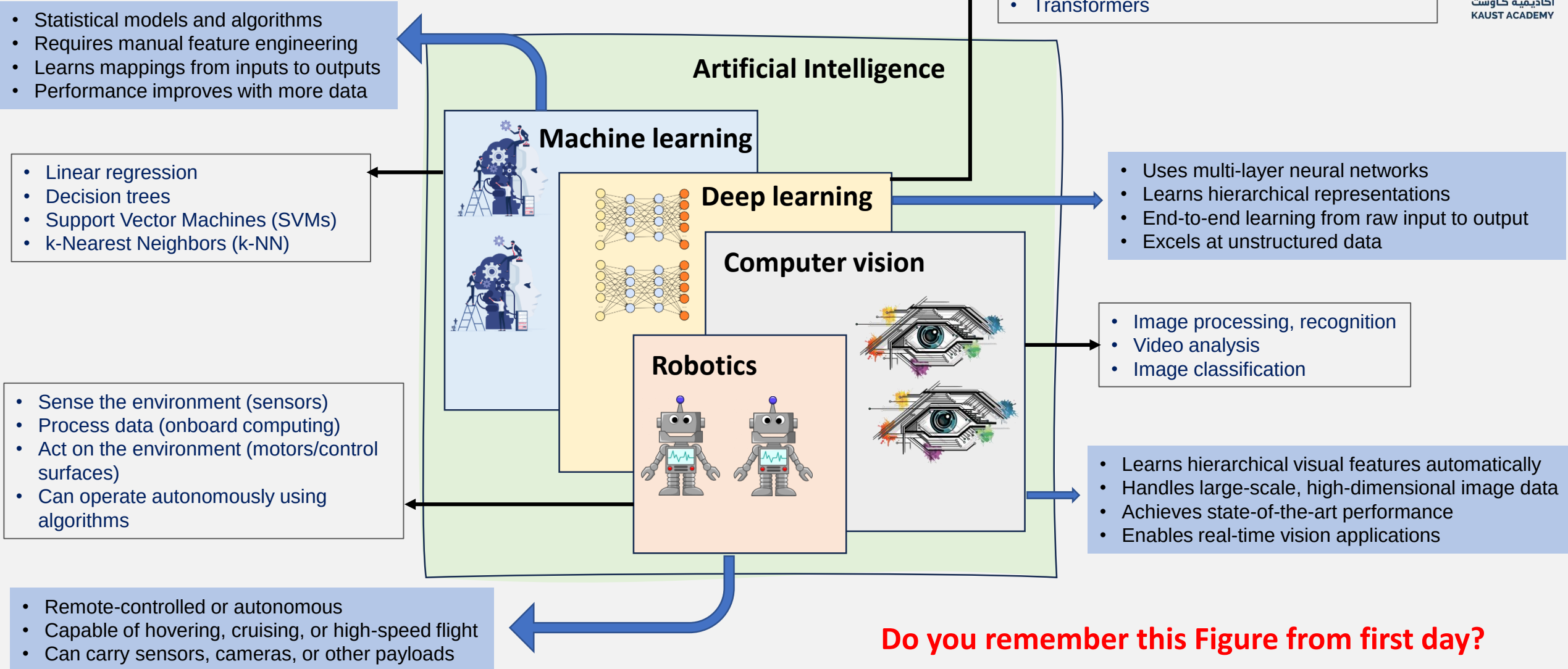
- How deep learning dominates modern AI
- Brief recap of traditional AI and ML concepts
- History of advanced AI (DL)
- What is Deep Learning
- What is Perceptron
- Abstract Neuron Model
- Multilayer Perceptrons (MLPs)
- Feedforward Neural Networks
- Backpropagation Algorithm and Gradient Decent
- Importance of Weight Initialization and Learning Rate



How deep learning dominates modern AI



Why deep learning dominates modern AI



Traditional AI and ML Recap

Key Points Recap

AI: Simulating human intelligence (rule-based systems, expert systems).

Machine Learning: Learning patterns from data (Linear regression, Decision trees).

Limitations of Traditional ML:

Feature engineering: Handcrafting features is tedious.

Performance: Struggles with high-dimensional data like images or text.



ML Recap

Basics of Machine Learning (ML)

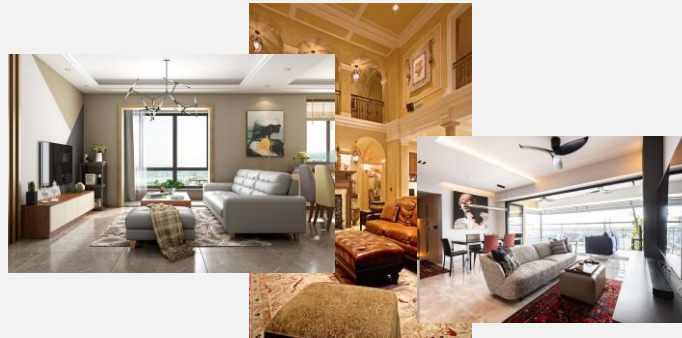
A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T** as measured by **P**, improves with experience **E**

Machine Learning, Tom Mitchell, 1997

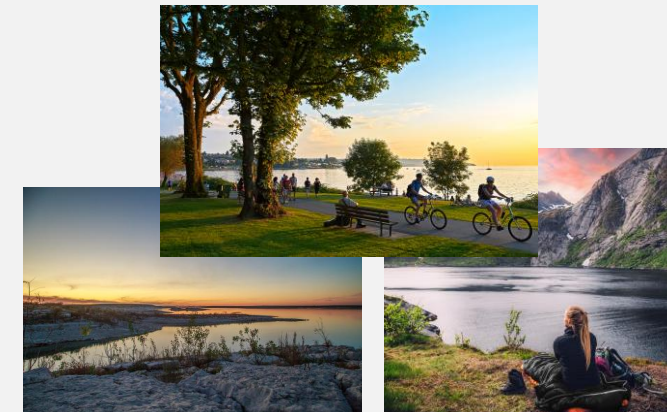
Example 1: Image Classification



Task: Determine if the image is indoor or outdoor
Performance measure: Probability of misclassification



Indoor



Outdoor

ML Recap

Basics of Machine Learning (ML)

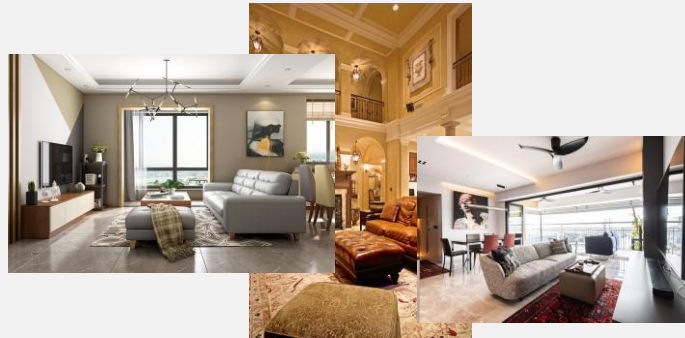
Example 1: Image classification

A few terminologies

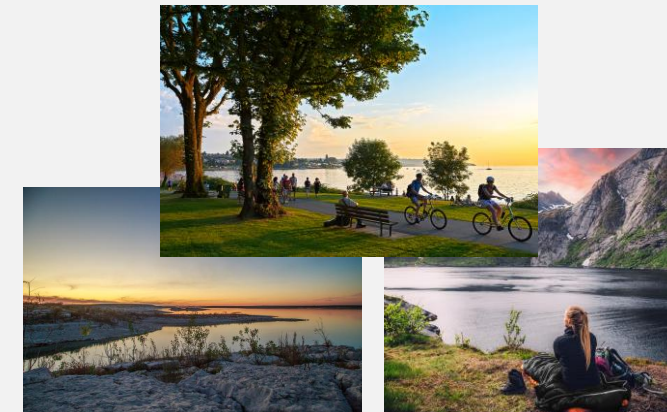
Training data: The images given for learning, **Test data:** The images to be classified

Binary classification: Classify into two classes, **Multi-class Classification:** ?

Example 1: Image Classification



Indoor



Outdoor

Task: Determine if the image is indoor or outdoor
Performance measure: Probability of misclassification

ML Recap

Basics of Machine Learning (ML)

Formulation



Indoor

Extract
Features

Color Histogram



Red Green Blue

Feature vector: x_i

Label: y_i

0

ML Recap

Basics of Machine Learning (ML)

Formulation

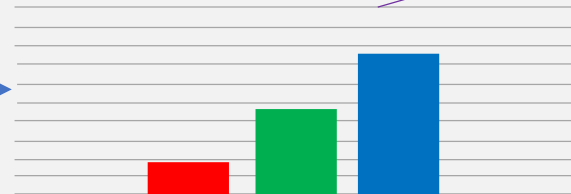


Outdoor

Extract

Features

Color Histogram



Red Green Blue

Feature vector: x_i

Label: y_i

1

Why Deep Learning?

Basic Steps in ML

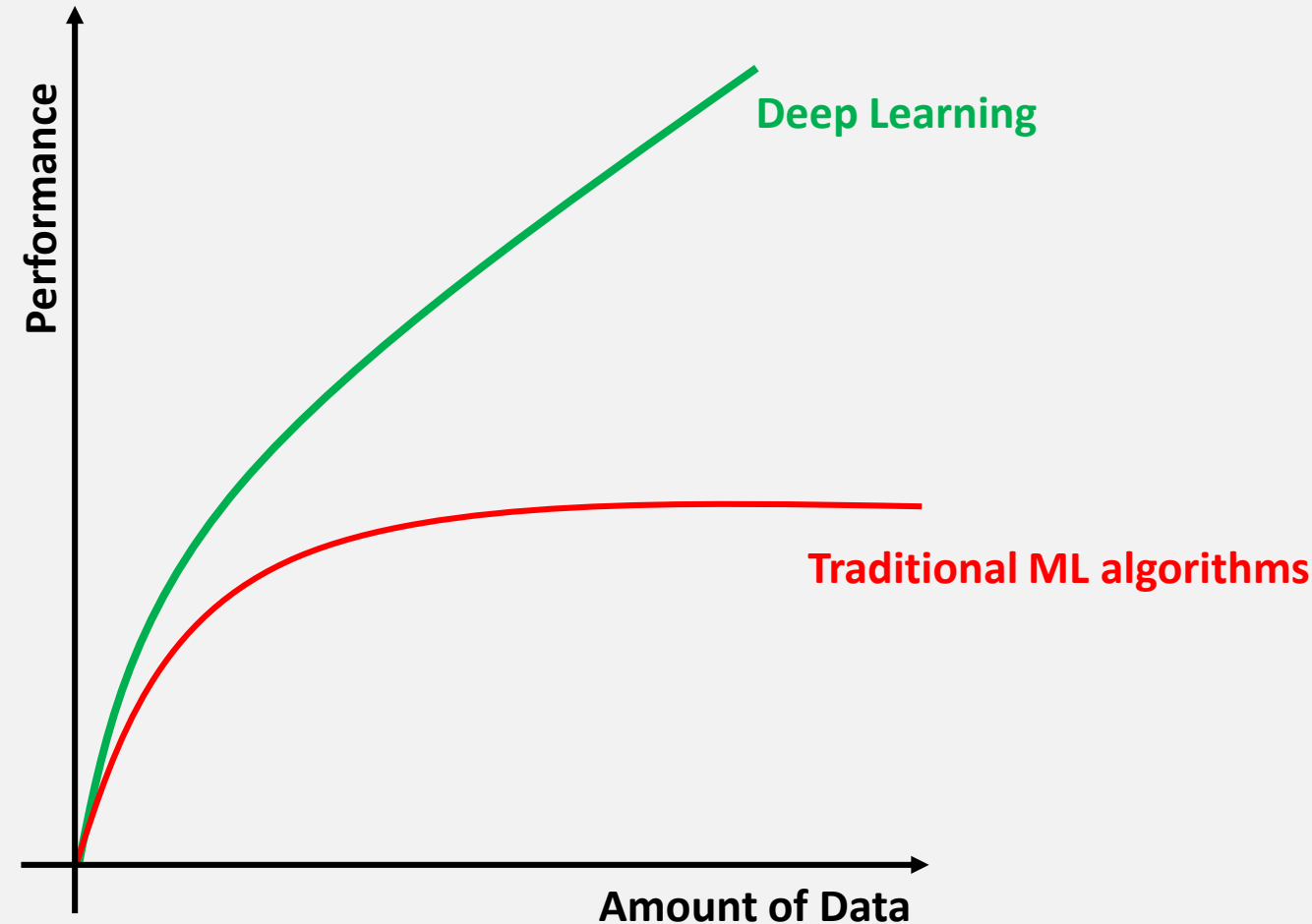
1. Collect data and extract features
2. Build model
3. Optimization: Minimize the error

DL: Requires large amounts of data

ML: Can be effective with smaller, structured datasets

DL: Can outperform machine learning for complex tasks like image recognition and natural language understanding

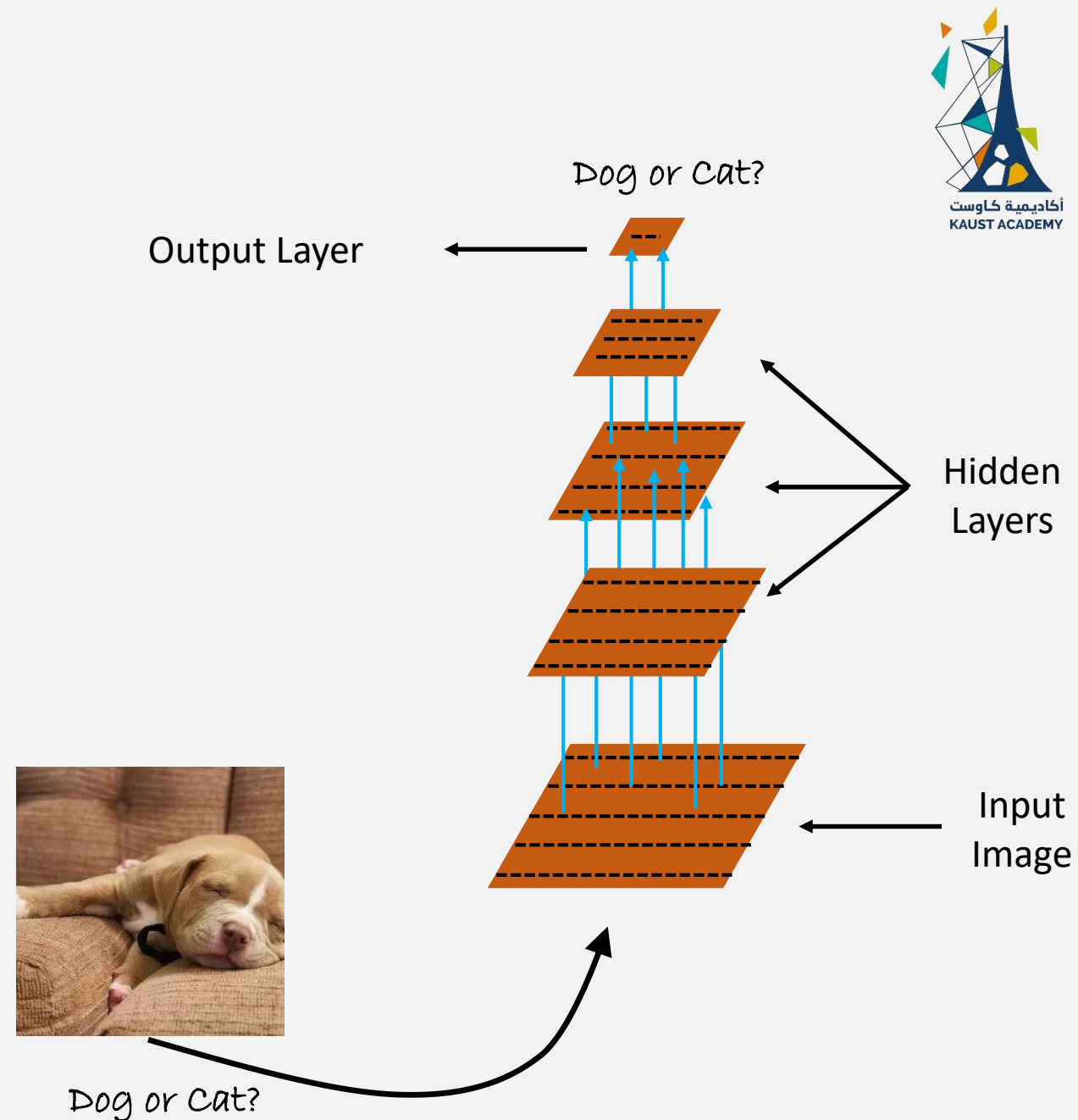
ML: Can be more efficient and easier to implement for simpler tasks



Deep Learning?

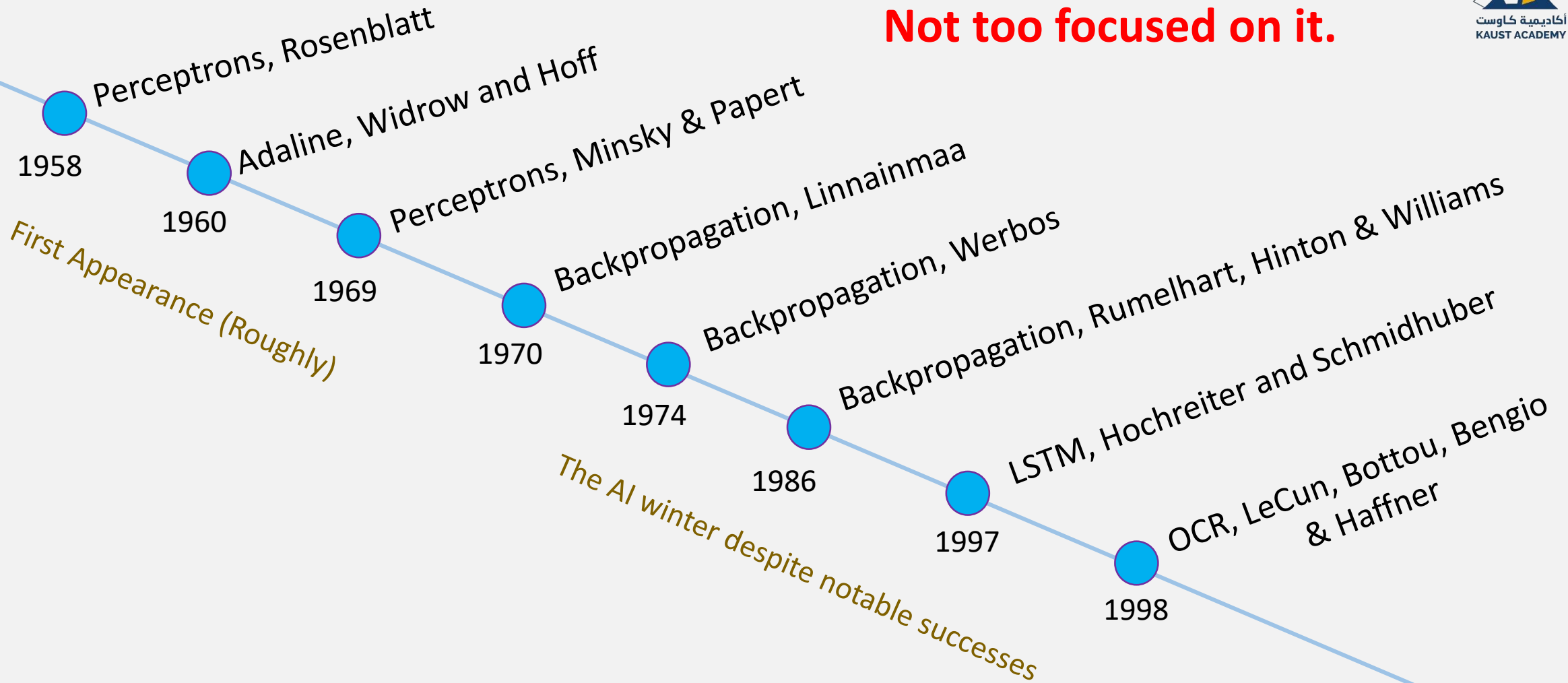
What is Deep Learning

- Sub-field of ML which attempts to learn high-level abstractions in data by utilizing hierarchical architectures.
- Using a neural network with several layers of nodes between the input and output
- The series of layers between input & output do **feature identification** and processing in a series of stages, just as our brains seem to.



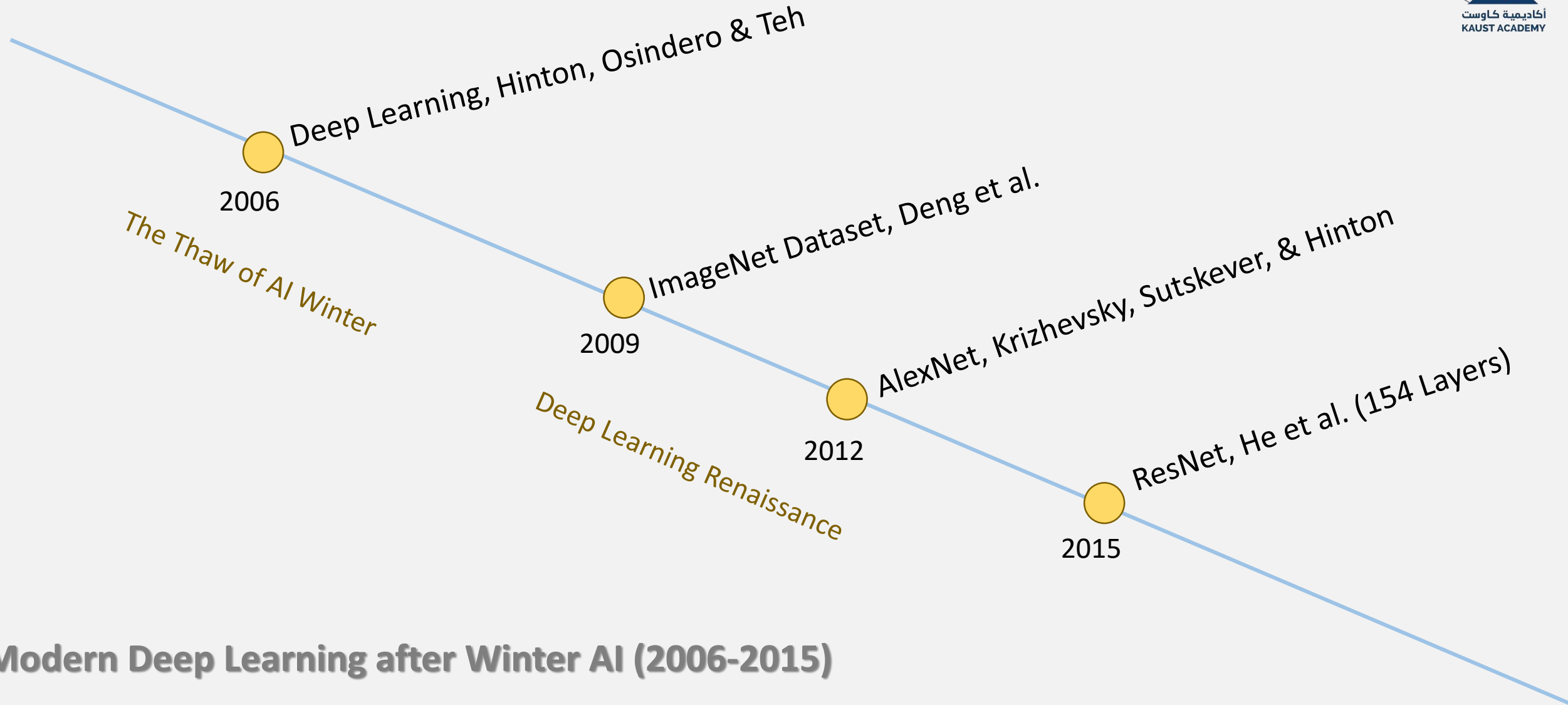
History of DL and Neural Network

Not too focused on it.

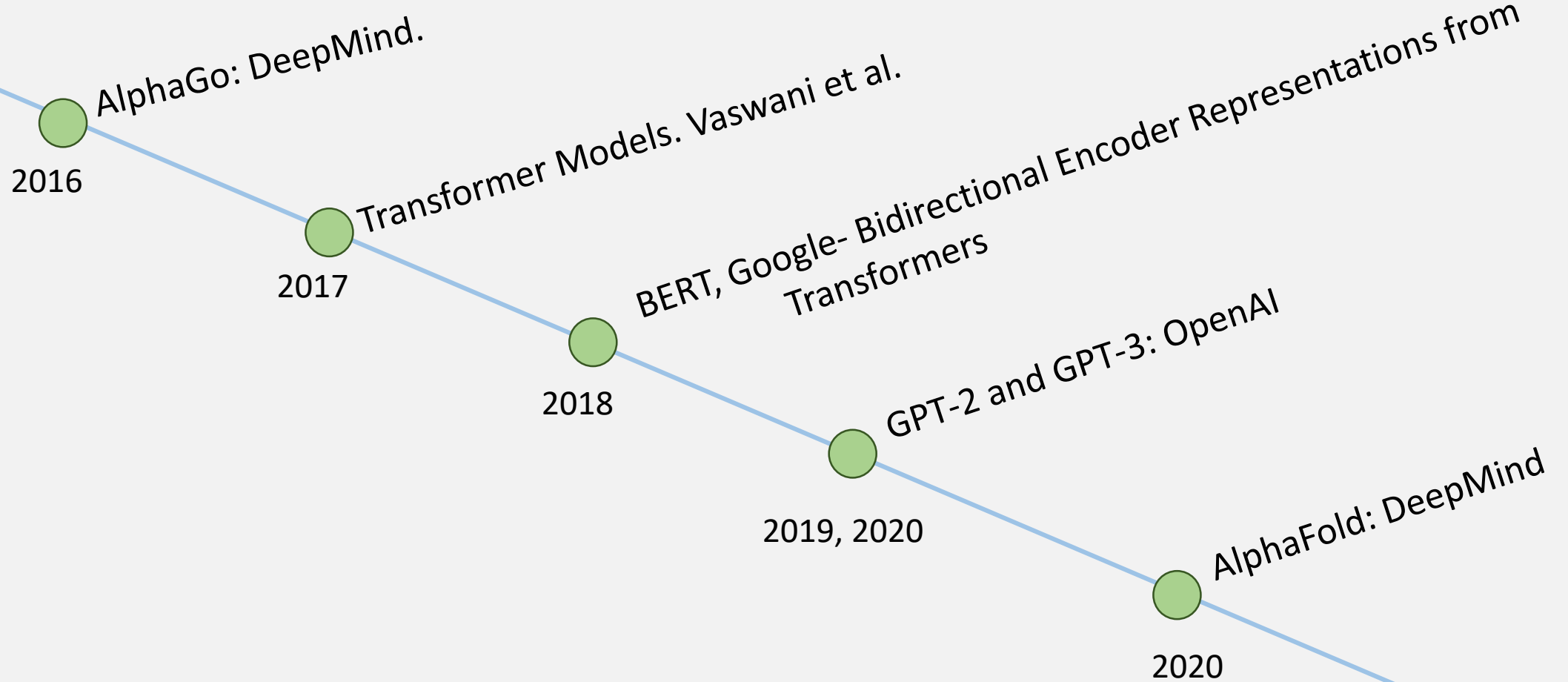


Early Foundations (1958–1998)

History of DL and Neural Network



History of DL and Neural Network



Advanced Deep Learning (2016-2020)

History of DL and Neural Network

2021–2025

2021: DALL-E – OpenAI

2022: Stable Diffusion – Stability AI

2022: ChatGPT – OpenAI

2023: GPT-4 – OpenAI

2023: DALL-E 3 – OpenAI

2023: Gemini – DeepMind

2024: LLaMA 4 – Meta

2025: GPT Image – OpenAI

2025: AI Scientific Advances – DeepMind & AlphaFold



Generative Model

From 2021 to 2025, the dominant trend in AI and deep learning is generative models.

What is Generative Modelling?

A generative model learns the underlying data distribution and can generate new, realistic data samples.

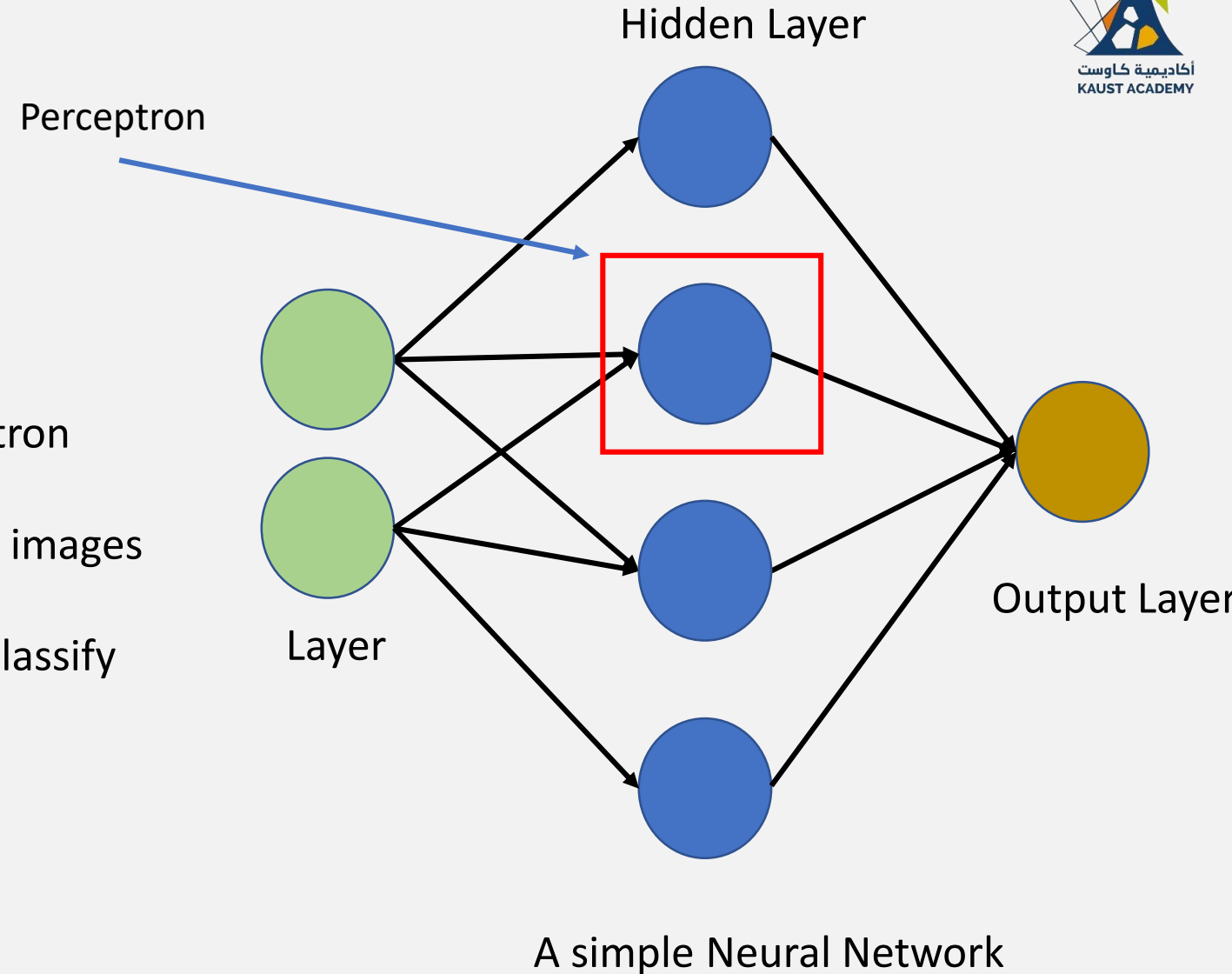
Instead of only predicting labels, generative models learn *how data is created*.

Aspect	Generative Models	Discriminative Models
Objective	Model data distribution	Model decision boundary
Output	New data samples	Labels or predictions
Learning Target	($p(x)$) or ($p(x,y)$)	($p(y$
Examples	GANs, VAEs, LSTMs, Transformers	CNN classifiers, SVMs

Perceptron

● Perceptrons, Rosenblatt 1957/58

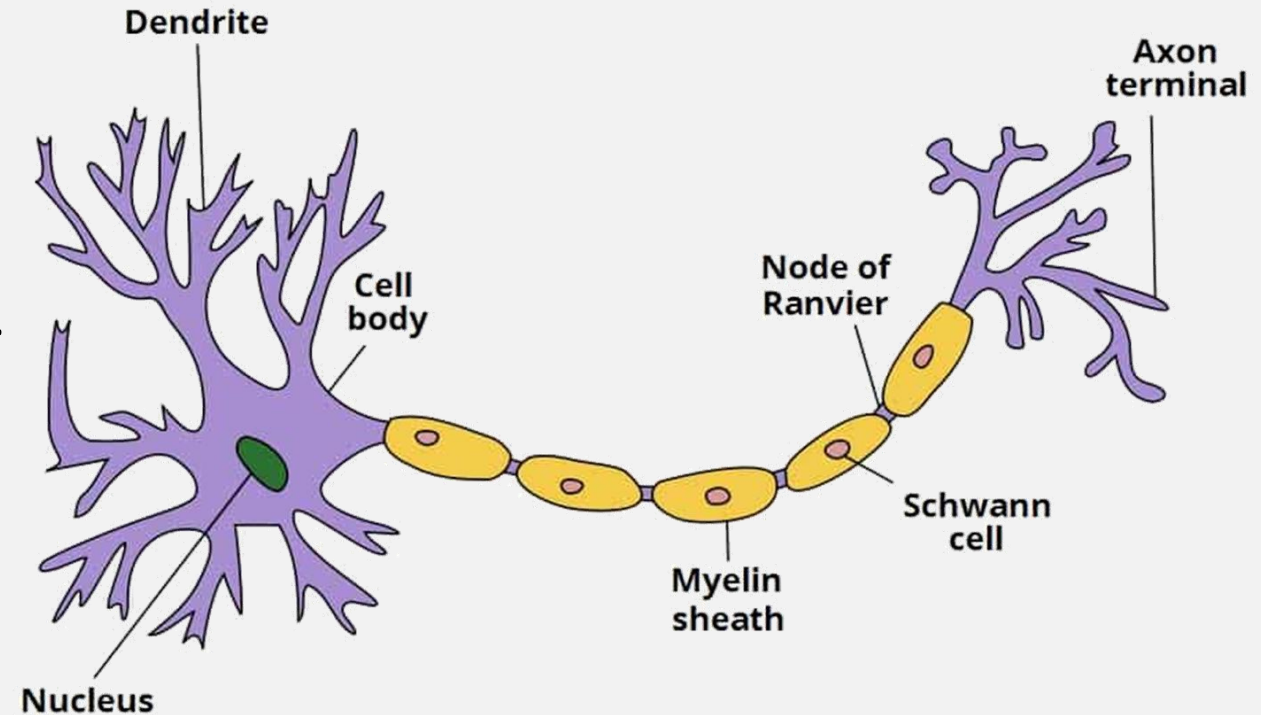
- Developed first implementation of perceptron
- Machine consist of 400 photocells, classify images
- Perceptron is one-layer network that can classify things into two parts



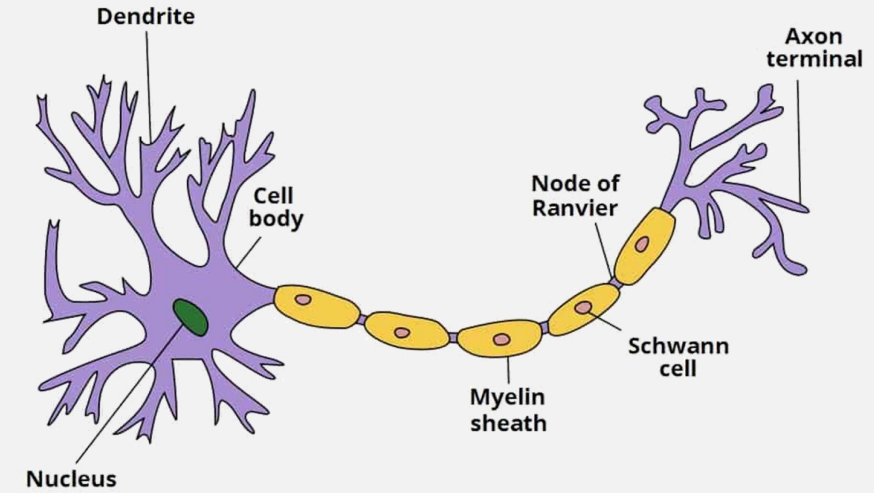
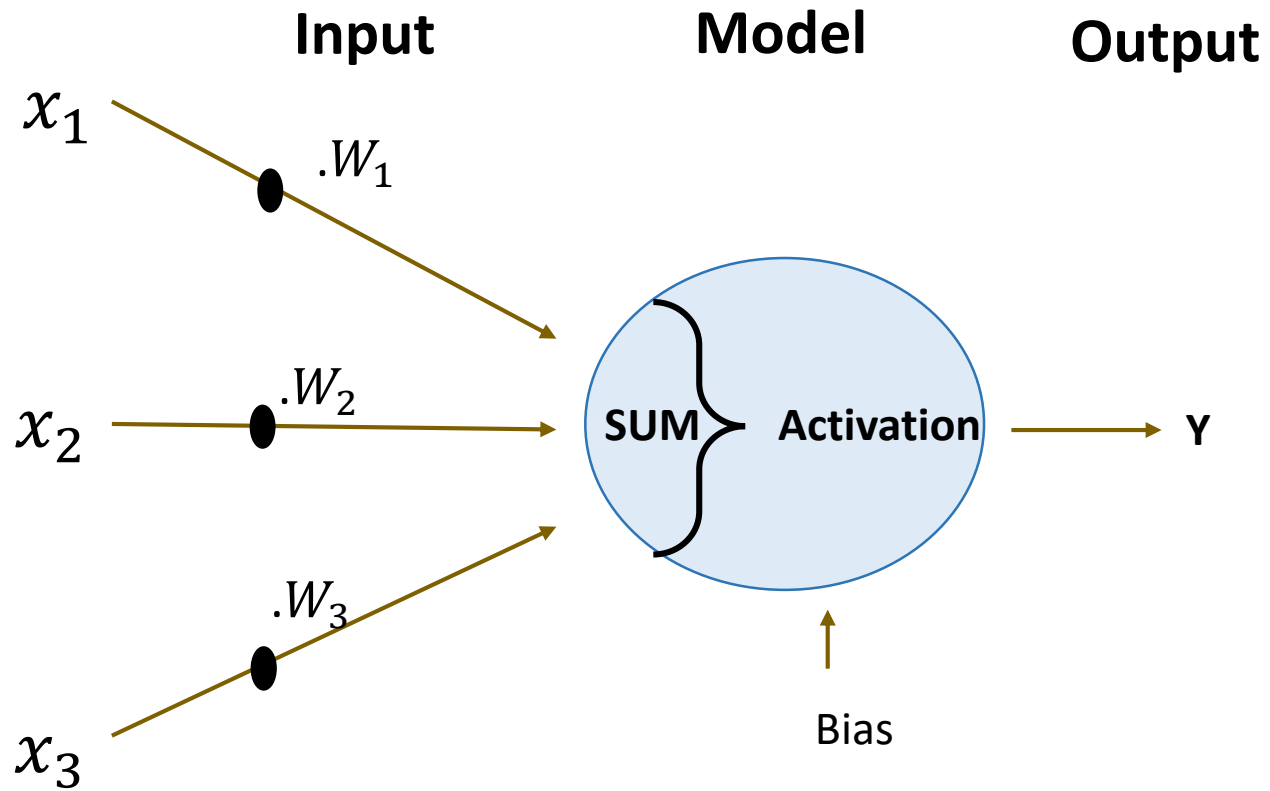
Perceptron

● Perceptrons, Rosenblatt 1957/58

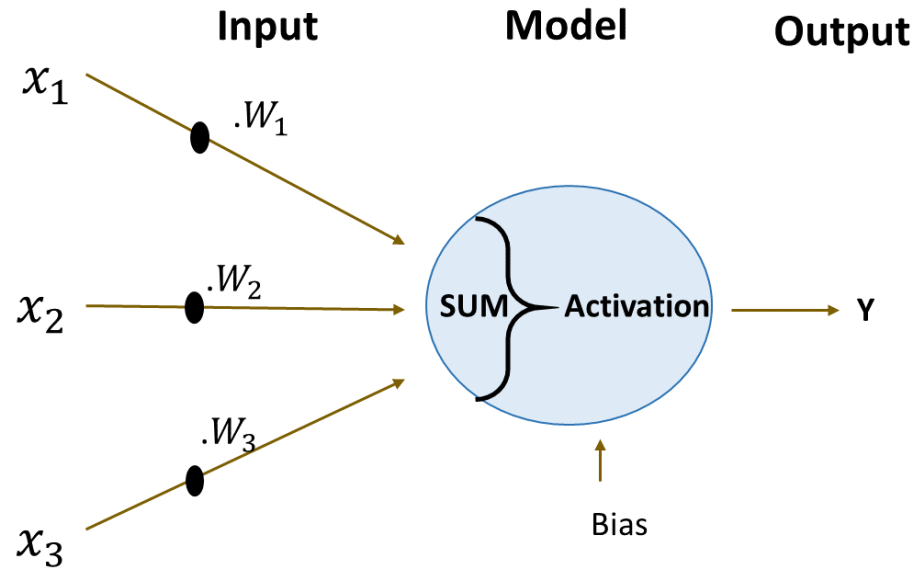
- Inspired by biological brain units called neurons.
- These neurons receive signals from other neurons.
- Depending on the strength, the neuron can either activate or off/stay.



Perceptron



Perceptron



Apply activation function to decide the output:
 $Y = \text{activation}(z)$

In a simple perceptron, the step function is commonly used.

$$z = W_1 \times x_1 + W_2 x_2 + W_3 x_3 + b$$

Where:

x_i : Input feature.

W_i : Weight for input x_i .

b : Bias

$$\text{Activation}(z) = \begin{cases} 1 & \text{if } z \geq 0 \text{ (class 1)} \\ 0 & \text{if } z < 0 \text{ (Class 0)} \end{cases}$$

Perceptron

Example: Pass or Fail Prediction

Scenario

Predict if a student will **Pass (1)** or **Fail (0)** an exam based on:

1. Hours studied (x_1),
2. Hours slept (x_2).

Given:

Weights: $W_1 = 0.6$, $W_2 = 0.4$

Bias: $b = -0.5$

Inputs: $x_1 = 4$ (hours studied), $x_2 = 3$ (hours slept).

$$z = W_1 \times x_1 + W_2 x_2 + b$$
$$Z = (0.6 \times 4) + (0.4 \times 3) + (-0.5)$$
$$Z = 2.4 + 1.2 - 0.5 = 3.1$$

Activation function: Since $z = 3.1 \geq 0$,
The output $y = 1$ (Pass)

```
[1] # Perceptron Function
def perceptron(inputs, weights, bias):
    weighted_sum = sum(x * w for x, w in zip(inputs, weights)) + bias
    return 1 if weighted_sum >= 0 else 0

# Inputs and Parameters
inputs = [4, 3] # Hours studied and slept
weights = [0.6, 0.4]
bias = -0.5

# Prediction
output = perceptron(inputs, weights, bias)
print("Prediction (Pass = 1, Fail = 0):", output)
```

Perceptron

Example: Pass or Fail Prediction

Scenario

Predict if a student will **Pass (1)** or **Fail (0)** an exam based on:

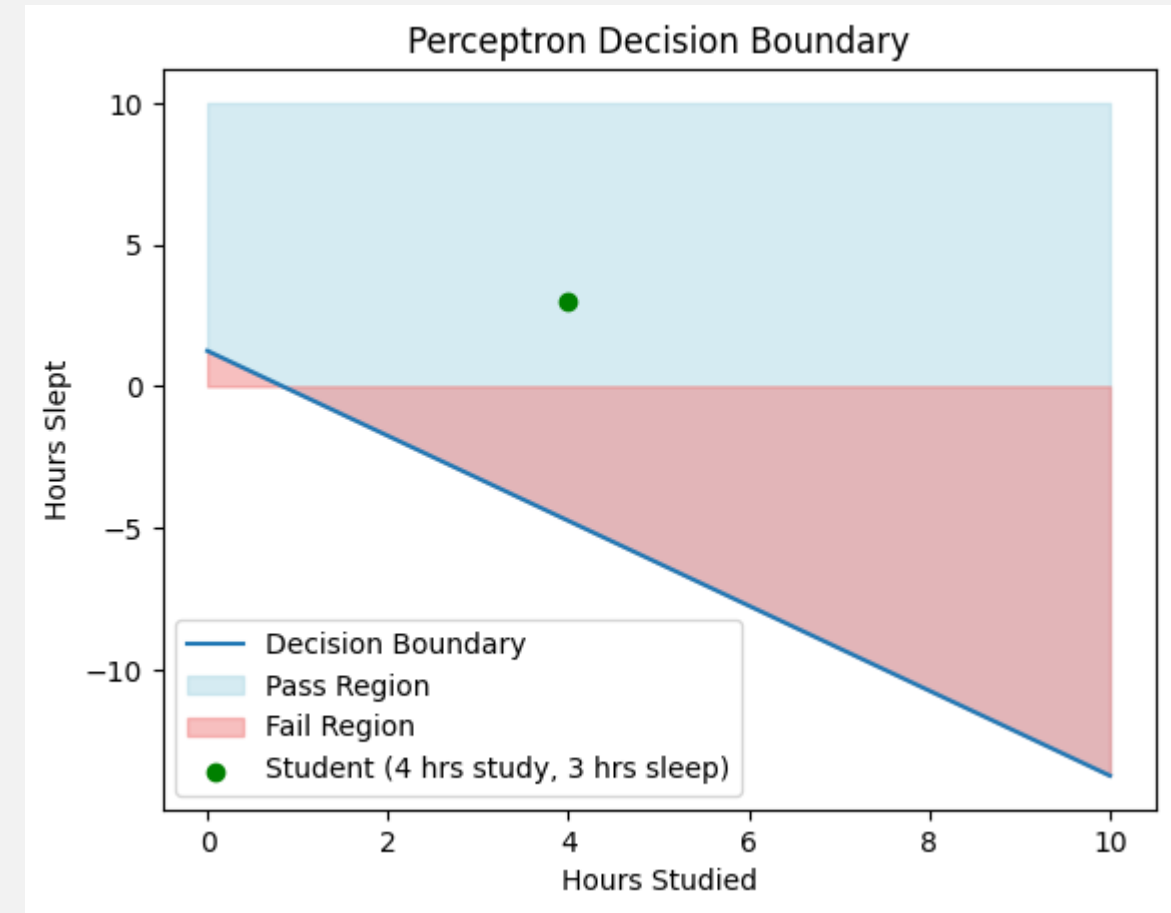
1. Hours studied (x_1), 2. Hours slept (x_2).

Given:

Weights: $W_1 = 0.6$, $W_2 = 0.4$

Bias: $b = -0.5$

Inputs: $x_1 = 4$ (hours studied), $x_2 = 3$ (hours slept).



Abstract Neuron Model

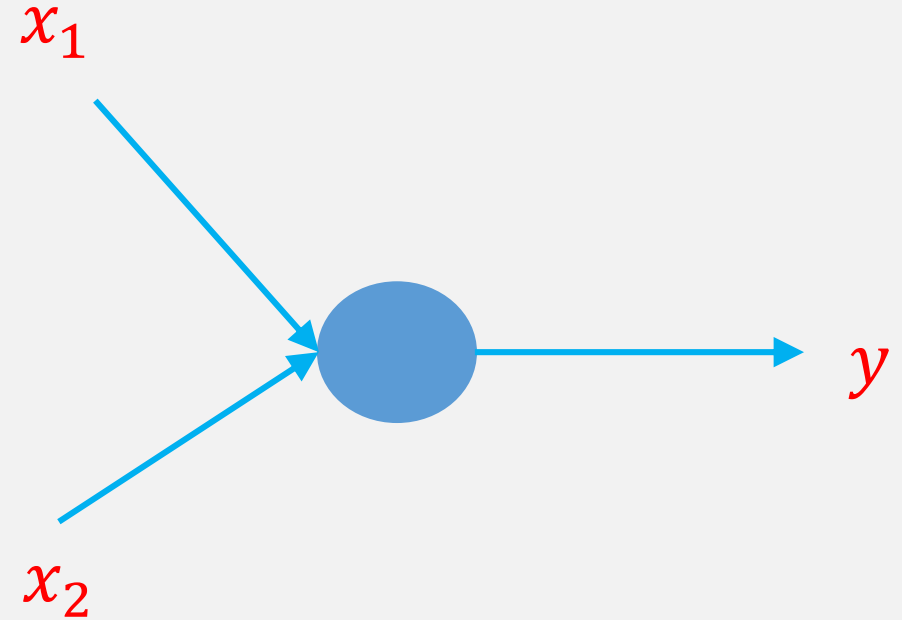
Neuron in a model processes information by taking **inputs**, **applying weights** to them, adding a **bias**, and then finding whether it should “activate” or not based on the result.

$$y = \theta^T x$$

x is the vector of inputs to the neuron,

θ is the vector of weights (also called a pattern or filter),

$\theta^T x$ is the dot product between weights and inputs.



$$y = \theta^T x + z$$

A bias is added to the weighted sum. This bias helps the neuron adjust its threshold for activation.

Abstract Neuron Model

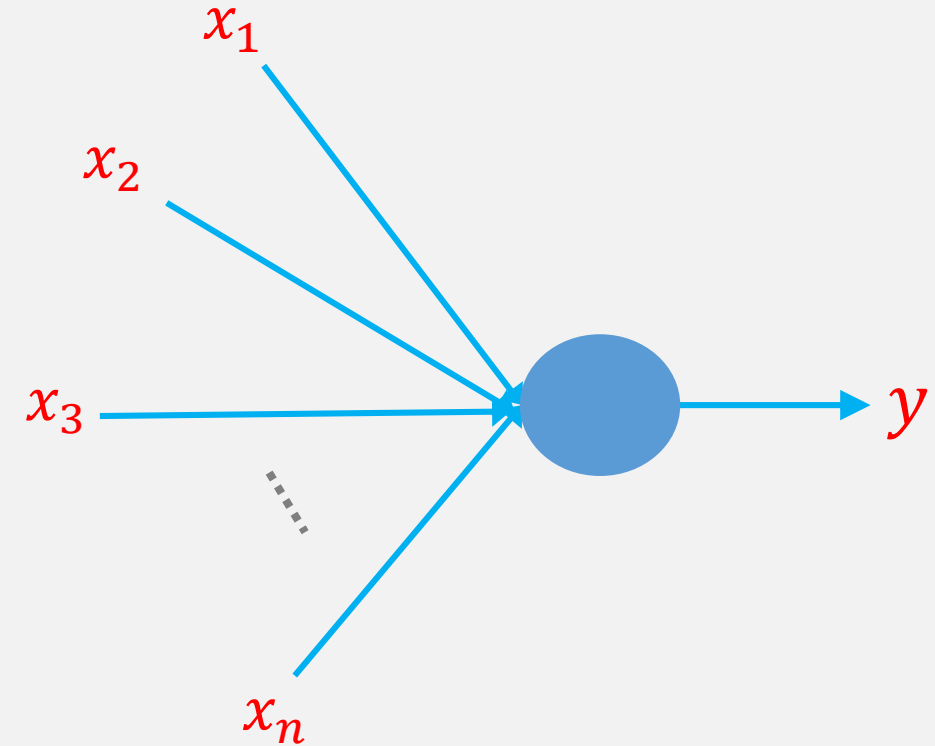
Neuron in a model processes information by taking **inputs**, **applying weights** to them, adding a **bias**, and then finding whether it should “activate” or not based on the result.

$$y = \theta^T x$$

x is the vector of inputs to the neuron,

θ is the vector of weights (also called a pattern or filter),

$\theta^T x$ is the dot product between weights and inputs.



$$y = \theta^T x + z$$

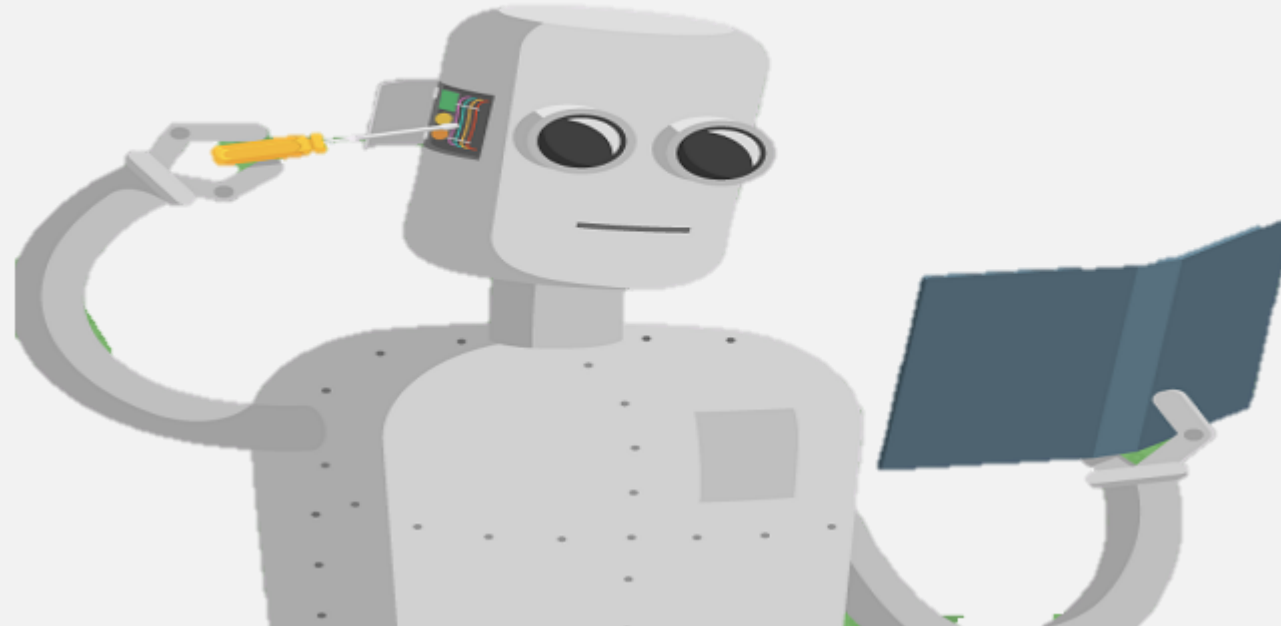
A bias is added to the weighted sum. This bias helps the neuron adjust its threshold for activation.

Artificial Neural Network

Problems with Single Perceptron?

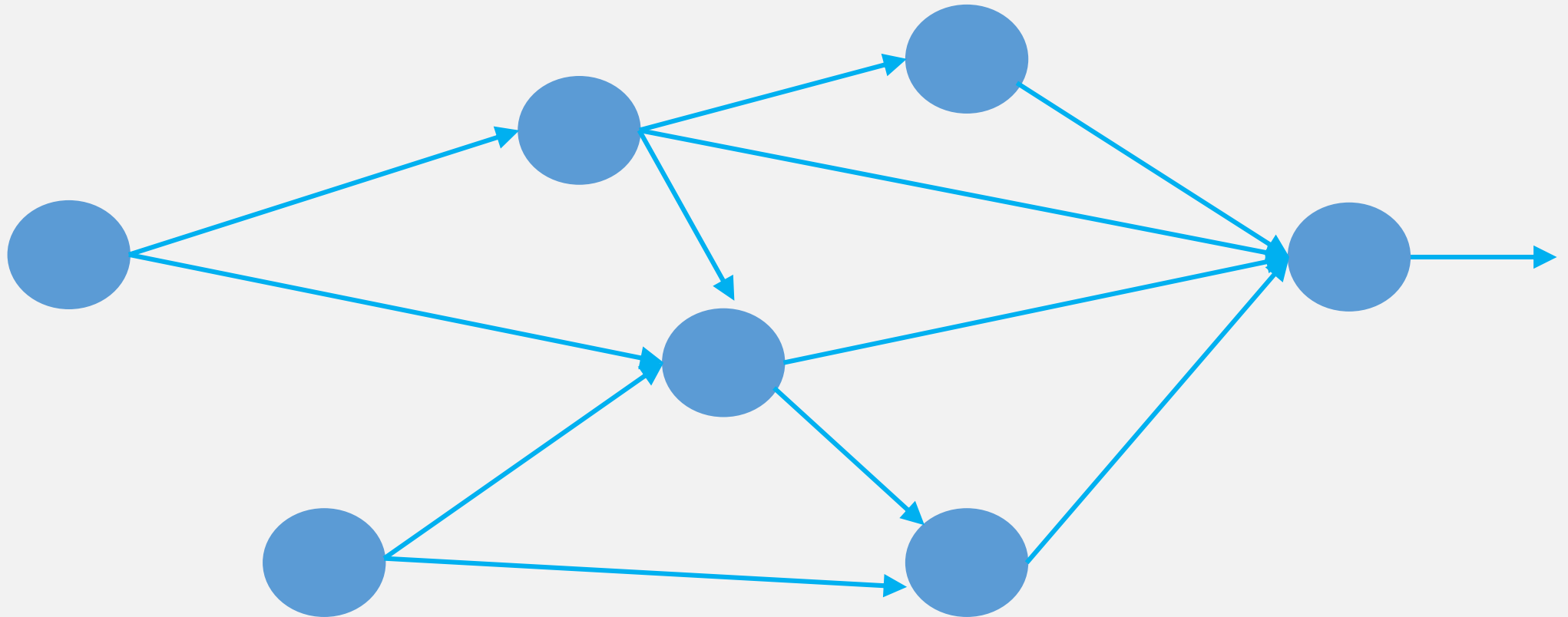
Single-layer perceptron can only solve linearly separable problems. (Step function, Linear function)

MLPs with hidden layers can approximate complex functions using non-linear activation functions.



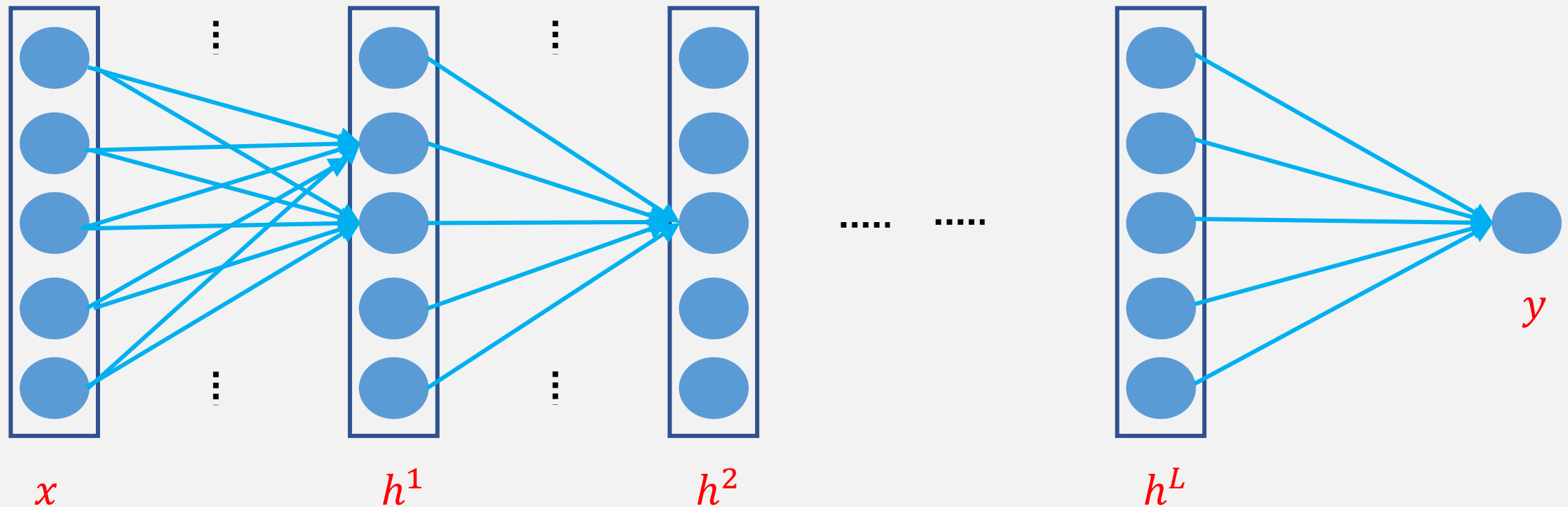
Artificial Neural Network

Combination of many neurons making MLP



Feedforward Propagation

Where data flows in one direction, from the input layer, through one or more hidden layers, to the output layer.



Feedforward Propagation

Where data flows in one direction, from the input layer, through one or more hidden layers, to the output layer.

Components

Representations:

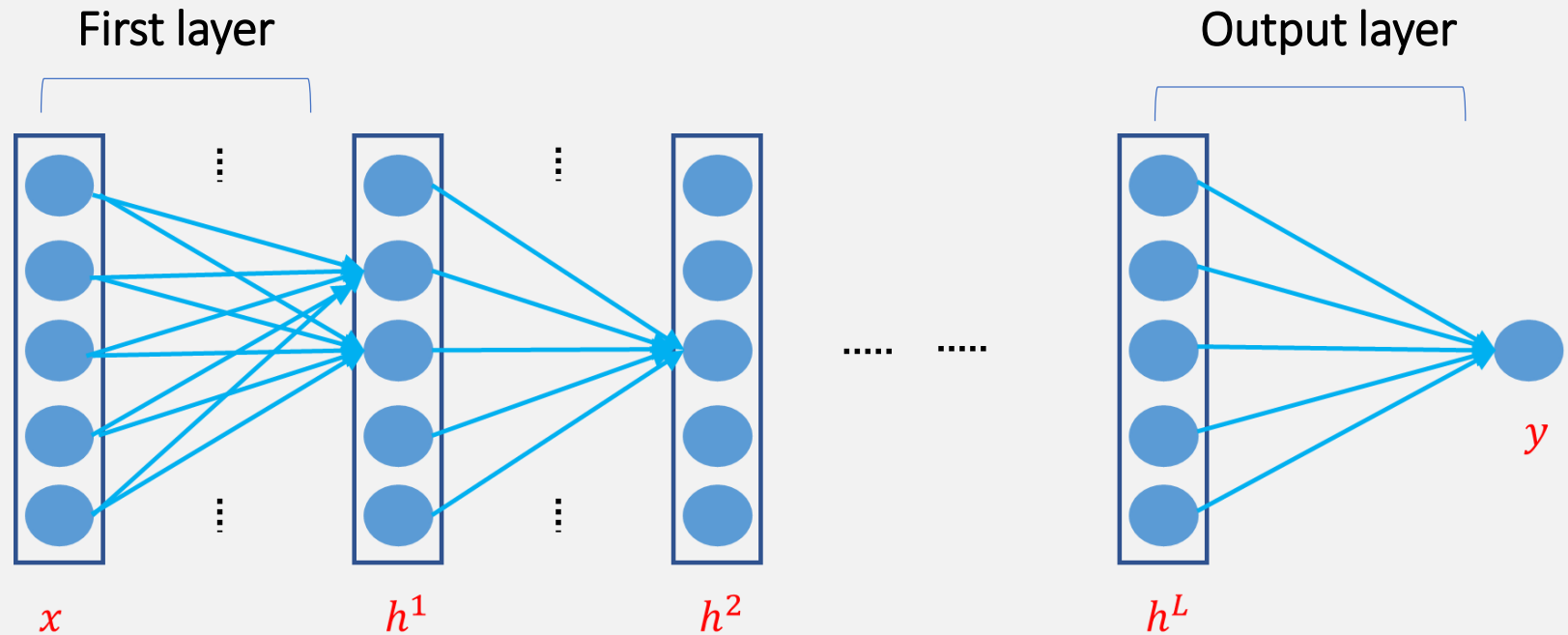
Input

Hidden variables

Layers/weights:

Hidden layers

Output layer



Feedforward Deep Network: Components

Input Layer

- The model receives **raw features** from the image.
- Represented as a vector



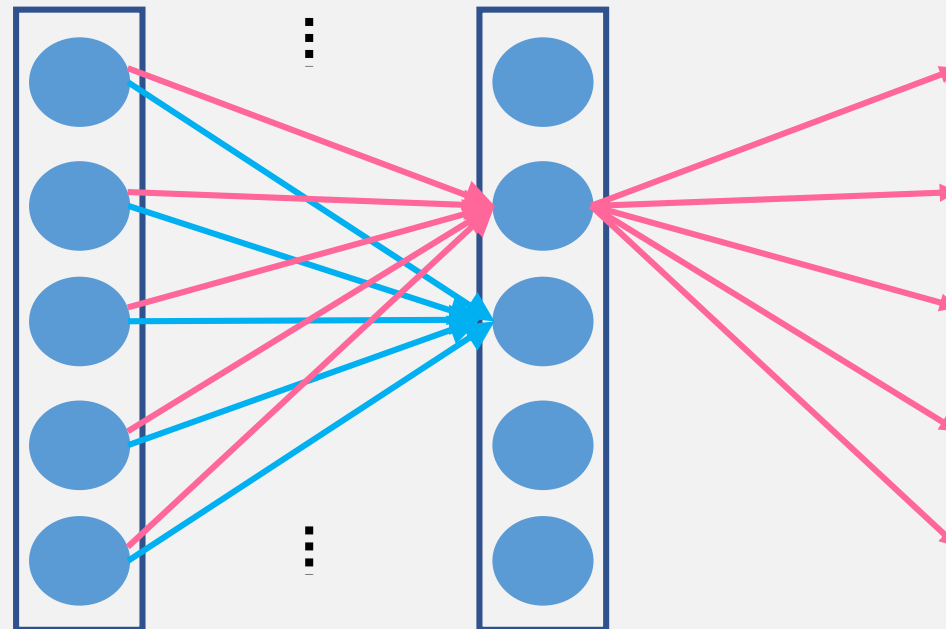
Expand



Feedforward Deep Network: Components

Hidden Layer

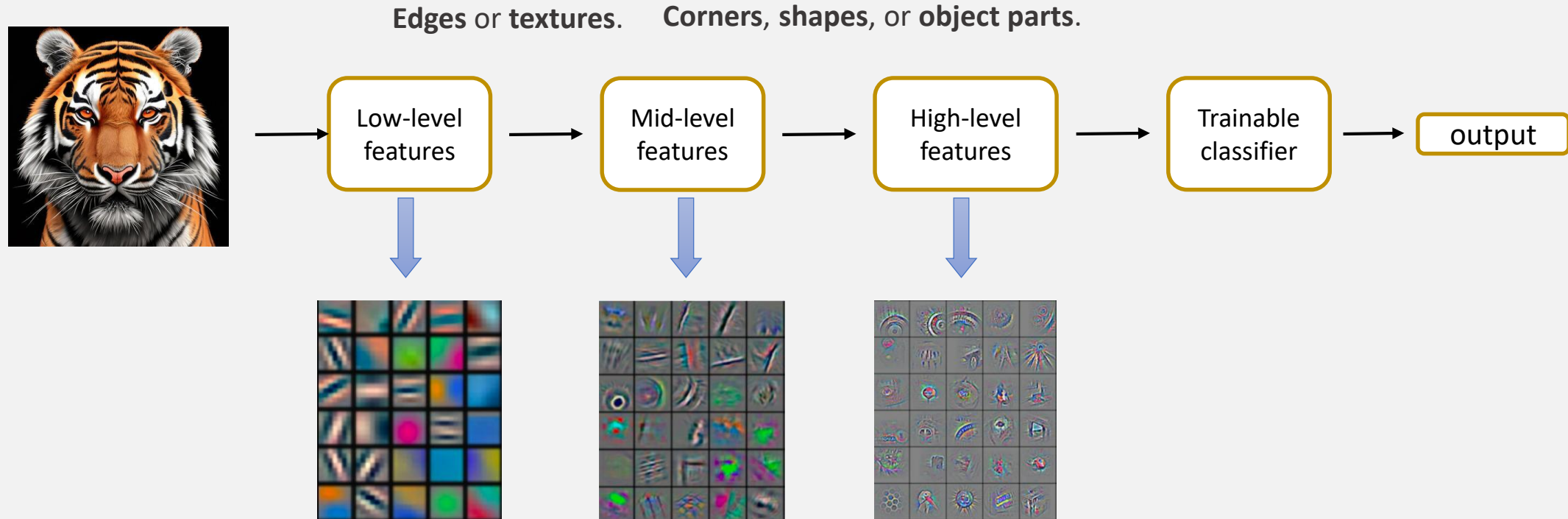
- Early hidden layers might detect low-level features like **edges**, **textures**, or **gradients**.
- Represented as a vector



Feedforward Deep Network: Components

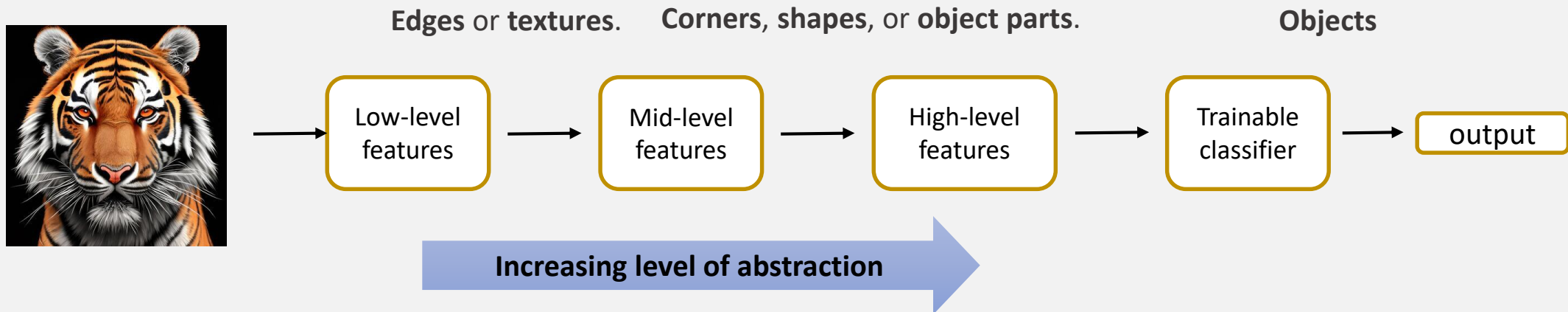
Hidden Layer

- Early hidden layers might detect low-level features like **edges**, **textures**, or **gradients**.
- Represented as a vector



Feedforward Deep Network: Components

Hidden Layer



Hierarchy of representations with increasing level of abstraction. Each stage is a kind of trainable nonlinear feature transform

Image recognition

Pixel → edge → texton → motif → part → object

Text

Character → word → word group → clause → sentence → story

Feedforward Deep Network: Components

Output Layer

The output layer produces the final prediction.

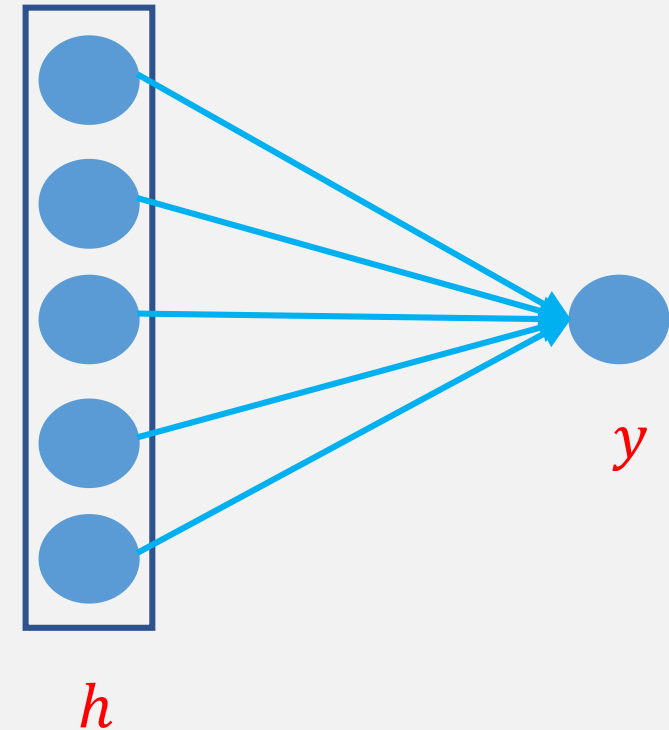
The number of neuron depends on the task.

Regression: 1 neuron with a linear activation function.

$$y = w^T h + b$$

Linear units: no nonlinearity

Example: Predict house prices.



Feedforward Deep Network: Components

Output Layer

The output layer produces the final prediction.

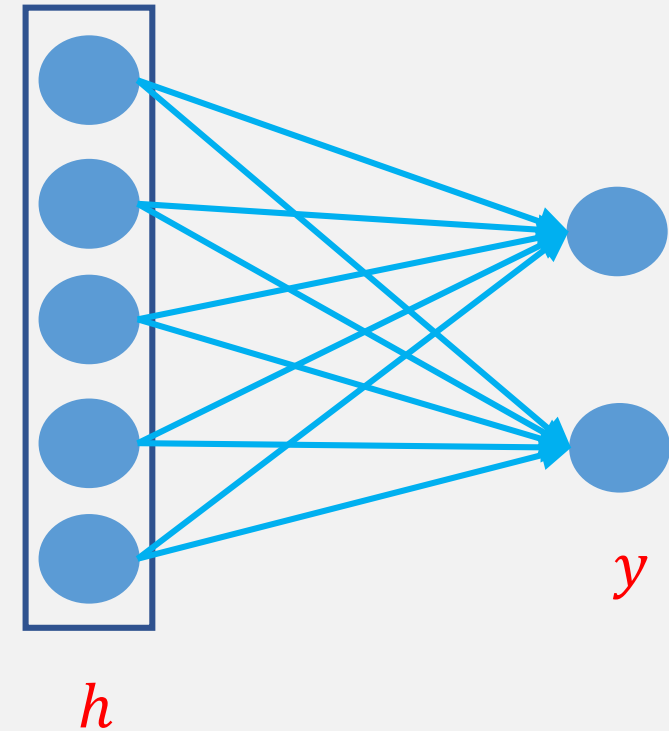
The number of neuron depends on the task.

Multi-Regression: n neuron with a linear activation function.

$$y = w^T h + b$$

Linear units: no nonlinearity

Example: Predict house prices.



Feedforward Deep Network: Components

Output Layer

The output layer produces the final prediction.

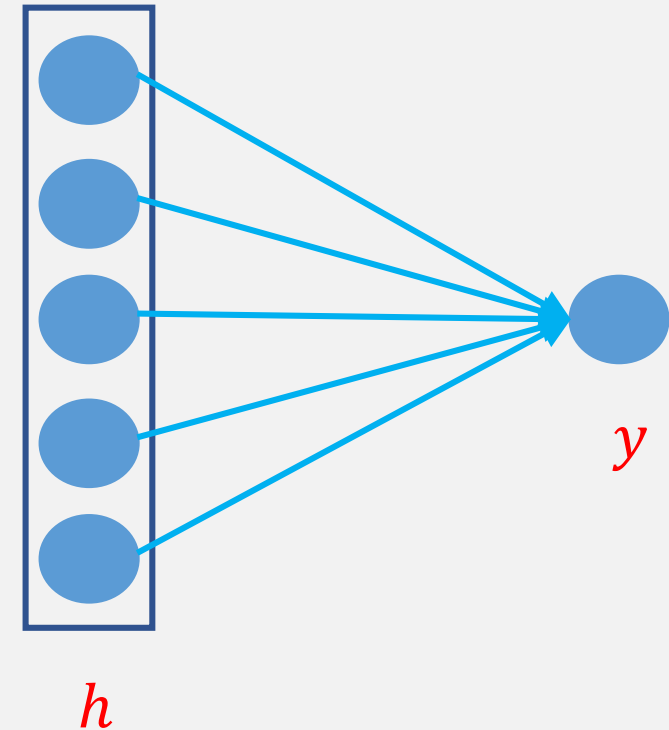
The number of neuron depends on the task.

Binary Classification: 1 neuron with a sigmoid activation function

$$y = \sigma(w^T h + b)$$

Linear units: no nonlinearity

Example: Approve loan (1) or deny loan (0).



Feedforward Deep Network: Components

Output Layer

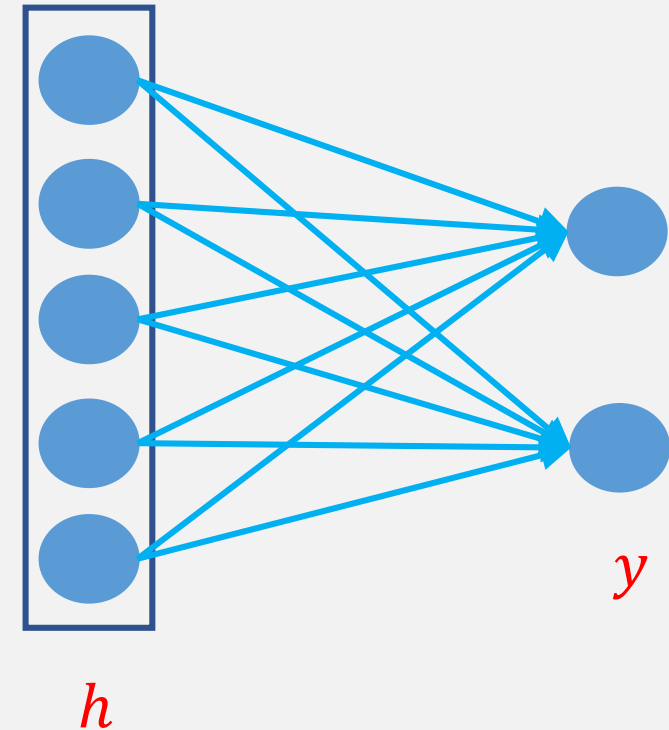
The output layer produces the final prediction.

The number of neuron depends on the task.

Multi-Class Classification: n neurons (one per class) with a softmax activation function.

$$y = \text{softmax}(z) \text{ where } z = W^T h + b$$

Example: Classify handwritten digits (0, 1, 2, ...9).



Applications of Deep Learning

Domain	Application Area	Specific Uses / Examples
Computer Vision	Image Classification	Object recognition, scene understanding
	Object Detection	Face detection, pedestrian detection
	Image Segmentation	Medical imaging, satellite image analysis
	Facial Recognition	Biometric security, phone unlocking
	Video Analysis	Surveillance, activity recognition
Natural Language Processing (NLP)	Machine Translation	Google Translate, language conversion
	Sentiment Analysis	Product reviews, social media monitoring
	Text Summarization	News summarization, document analysis
	Chatbots	Customer support, virtual assistants
	Question Answering	Search engines, AI tutors
Speech & Audio Processing	Speech Recognition	Voice commands, dictation software
	Text-to-Speech	Audiobooks, assistive technologies
	Speaker Identification	Call center authentication
	Audio Classification	Music genre detection
Healthcare	Medical Imaging	Tumor detection, X-ray analysis
	Disease Prediction	Cancer, heart disease prediction
	Drug Discovery	Molecular modeling
	Personalized Medicine	Treatment recommendations
	Health Monitoring	Wearable devices, ECG analysis

Applications of Deep Learning

Domain	Application Area	Specific Uses / Examples
Agriculture	Crop Disease Detection	Leaf disease identification
	Yield Prediction	Crop production forecasting
	Smart Irrigation	Water usage optimization
Security	Biometric Systems	Fingerprint, iris recognition
	Surveillance	Suspicious activity detection
	Cybersecurity	Malware detection
Gaming & Entertainment	Game AI	AlphaGo, NPC behavior
	Content Generation	Music, art, video generation
	Virtual Reality	Immersive environments
Education	Intelligent Tutoring	Personalized learning
	Automated Grading	Exam evaluation
	Learning Analytics	Student performance prediction
Transportation	Traffic Prediction	Congestion management
	Route Optimization	GPS navigation systems
Energy	Power Load Forecasting	Smart grids
	Fault Detection	Power system monitoring
Natural Sciences	Climate Modeling	Weather prediction
	Astronomy	Galaxy classification
Social Media	Content Moderation	Hate speech detection
	Trend Analysis	Viral content prediction

Applications of Deep Learning

Domain	Application Area	Specific Uses / Examples
Autonomous Systems	Self-driving Cars	Lane detection, traffic sign recognition
	Drones	Navigation, aerial surveillance
	Robotics Navigation	Path planning, obstacle avoidance
Finance	Fraud Detection	Credit card fraud prevention
	Algorithmic Trading	Stock market prediction
	Credit Scoring	Loan approval systems
	Risk Management	Financial forecasting
Retail & E-commerce	Recommendation Systems	Product and content recommendations
	Demand Forecasting	Inventory management
	Price Optimization	Dynamic pricing
	Customer Analytics	Buying behavior prediction
Manufacturing	Quality Control	Defect detection
	Predictive Maintenance	Machine failure prediction
	Automation	Smart factories

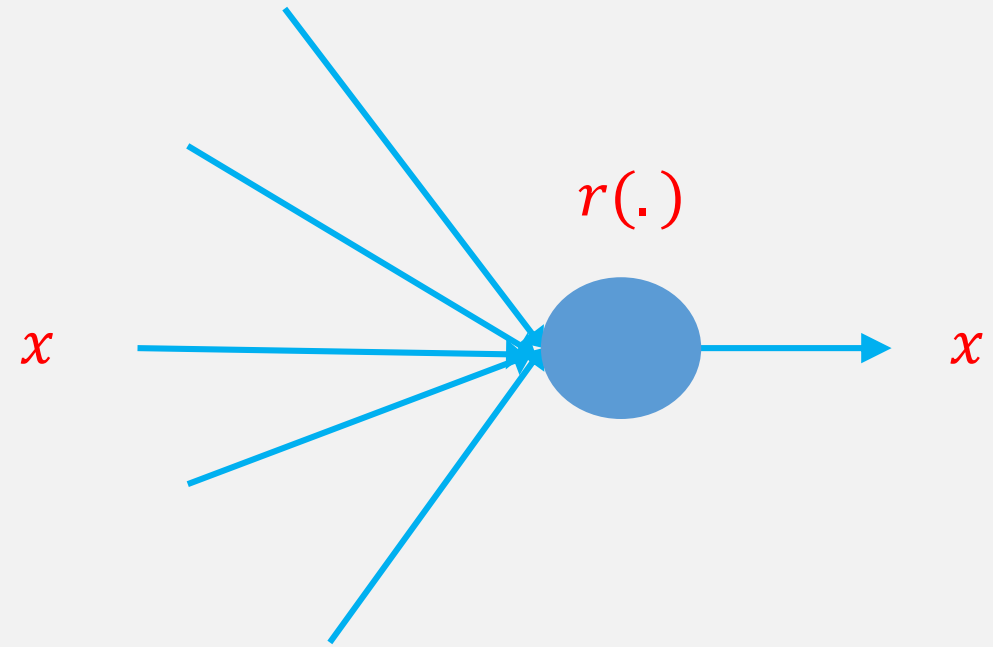
Feedforward Deep Network: Components

Activation Functions

- Consider a neuron.

$$Y = \sum(\text{weight} * \text{input}) + \text{bias}$$

- Now, the value of Y can be anything ranging from $-\infty$ to $+\infty$.
- The neuron really doesn't know the bounds of the value.
- So how do we decide whether the neuron should fire or not?



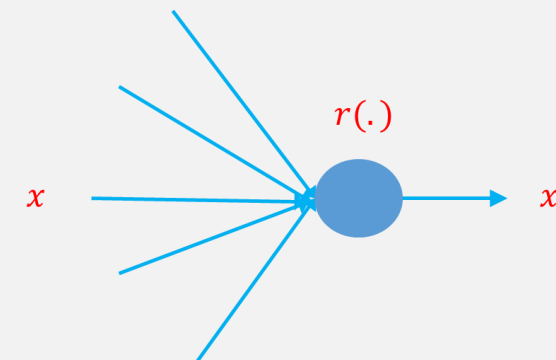
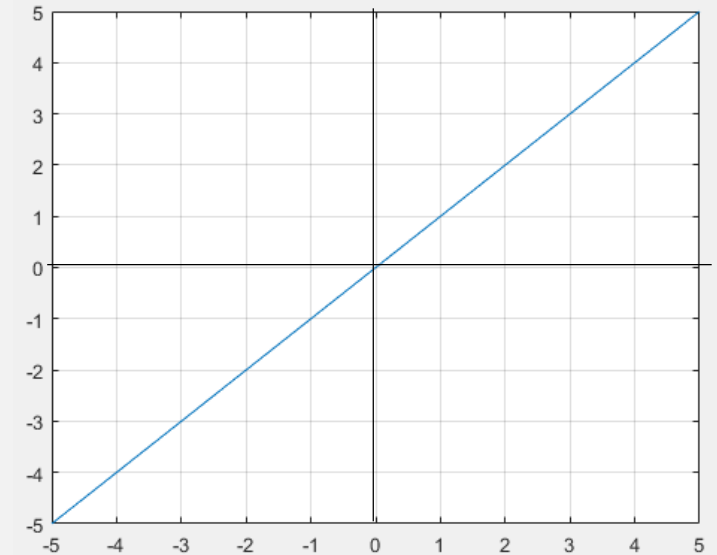
Feedforward Deep Network: Components

Activation Functions: Linear

- A straight line function where activation is proportional to input

$$f(x) = cx$$

- Not appropriate for modeling complex nonlinear functions.



Feedforward Deep Network: Components

Activation Functions: Sigmoid

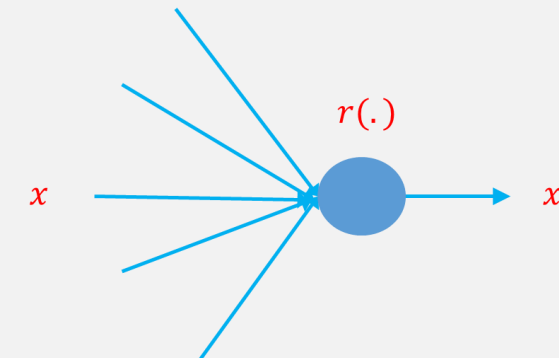
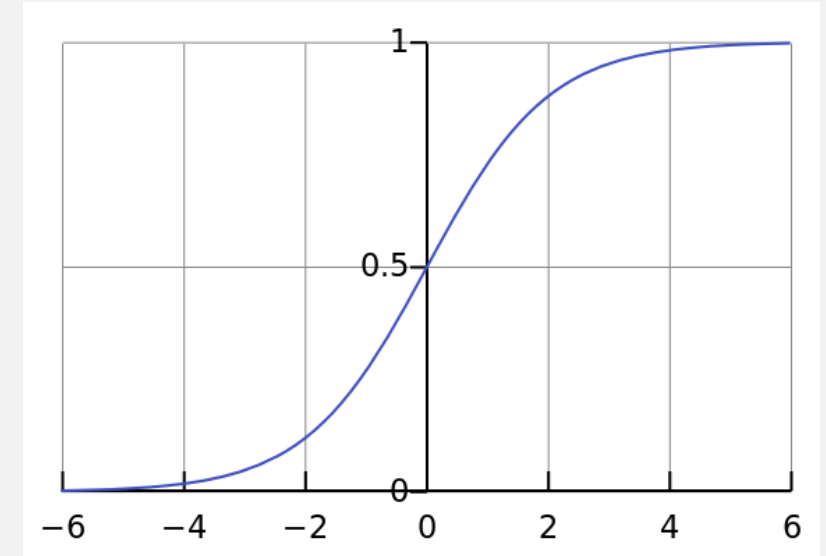
- If the value of Y is above a certain value, declare it “activated”.

$$f(x) = \frac{1}{1 + e^{-x}}$$

If sigmoid $(x) \geq 0.5$, predict class 1.

If sigmoid $(x) < 0.5$, predict class 0.

- Often used in binary classification tasks, where a threshold of 0.5 is used to make decisions.



Feedforward Deep Network: Components

Activation Functions: ReLu (Rectified Linear Unit)

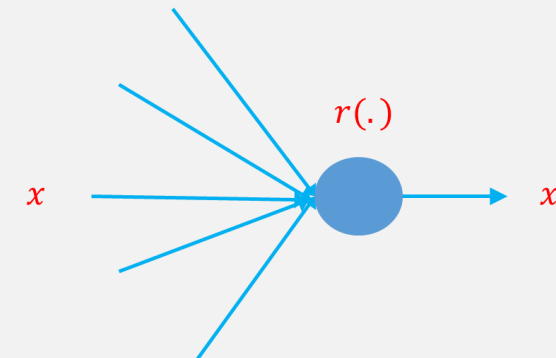
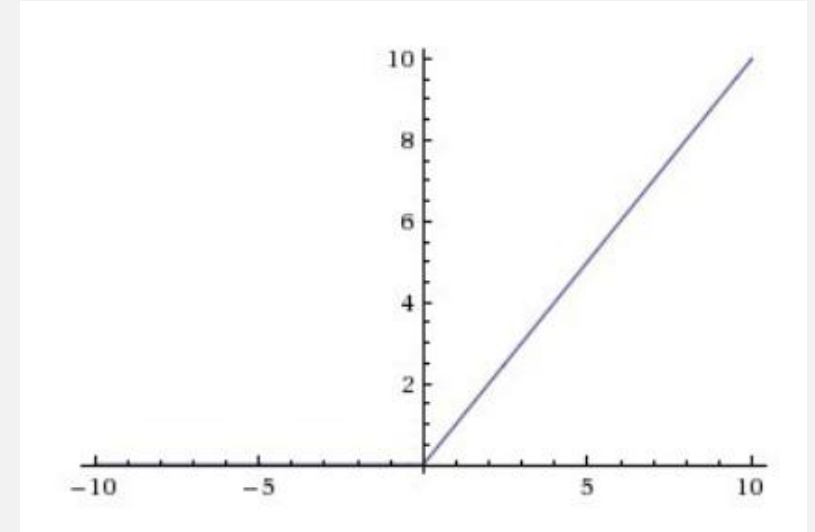
- It is a non-linear function and is capable of modeling complex functions.

$$f(x) = \max(0, x)$$

For any input x greater than 0, the output is x (no change).

For any input x less than or equal to 0, the output is 0 (the neuron is "turned off").

- The range of ReLu is $[0, \infty]$
- It ensures sparsity of activations



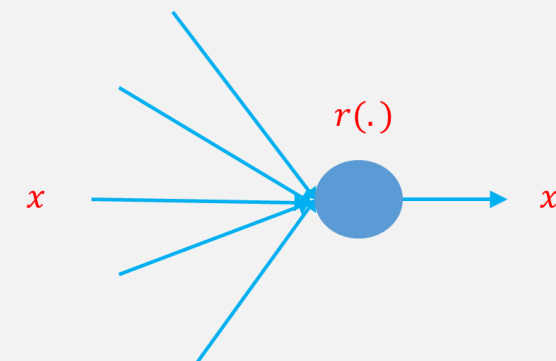
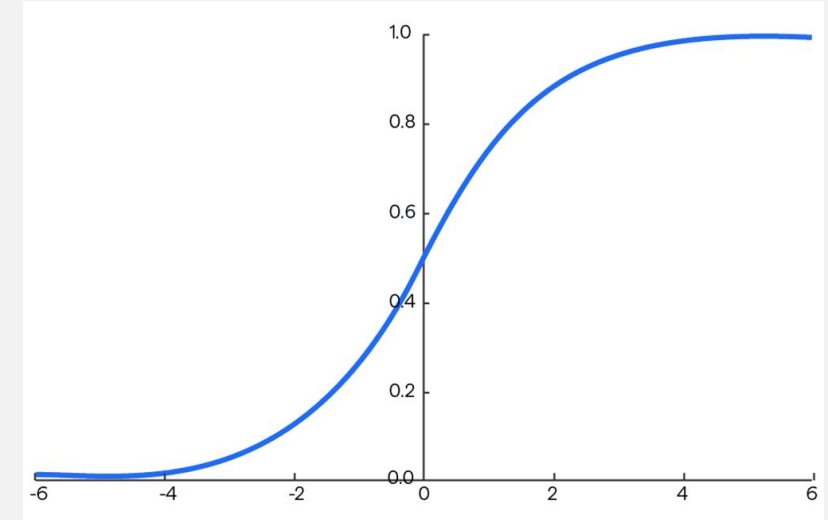
Feedforward Deep Network: Components

Activation Functions: Softmax

- In **softmax**, the output is a vector of probabilities, and there isn't a single threshold used in the same way as the sigmoid.
- **Thresholding** can be applied during **classification**:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

- For multi-class classification, after applying softmax, you typically select the class with the highest probability as the prediction.

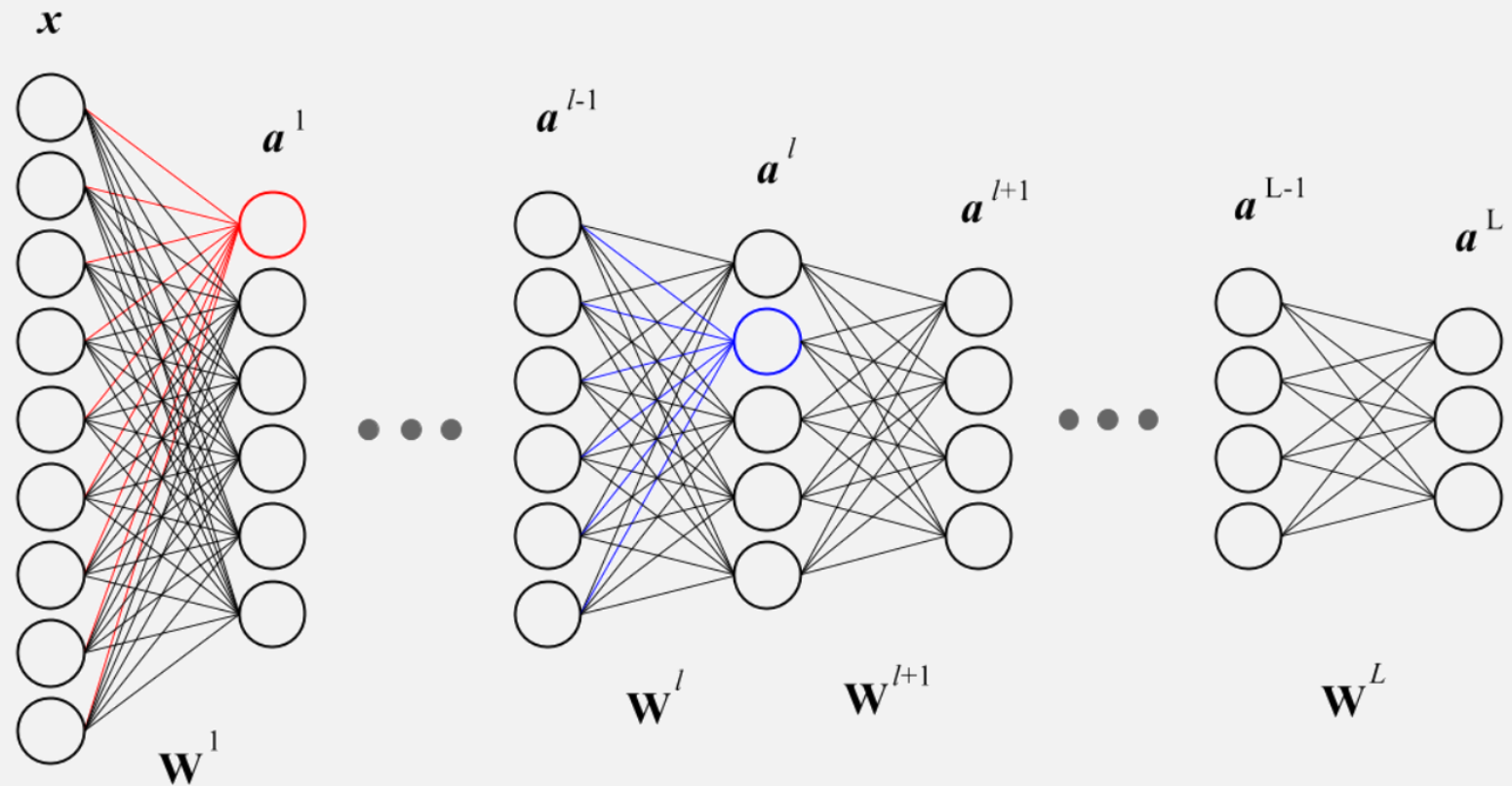


Forward Propagation

Problem:

MLPs have many weights and biases.

How do we update them to reduce the error?



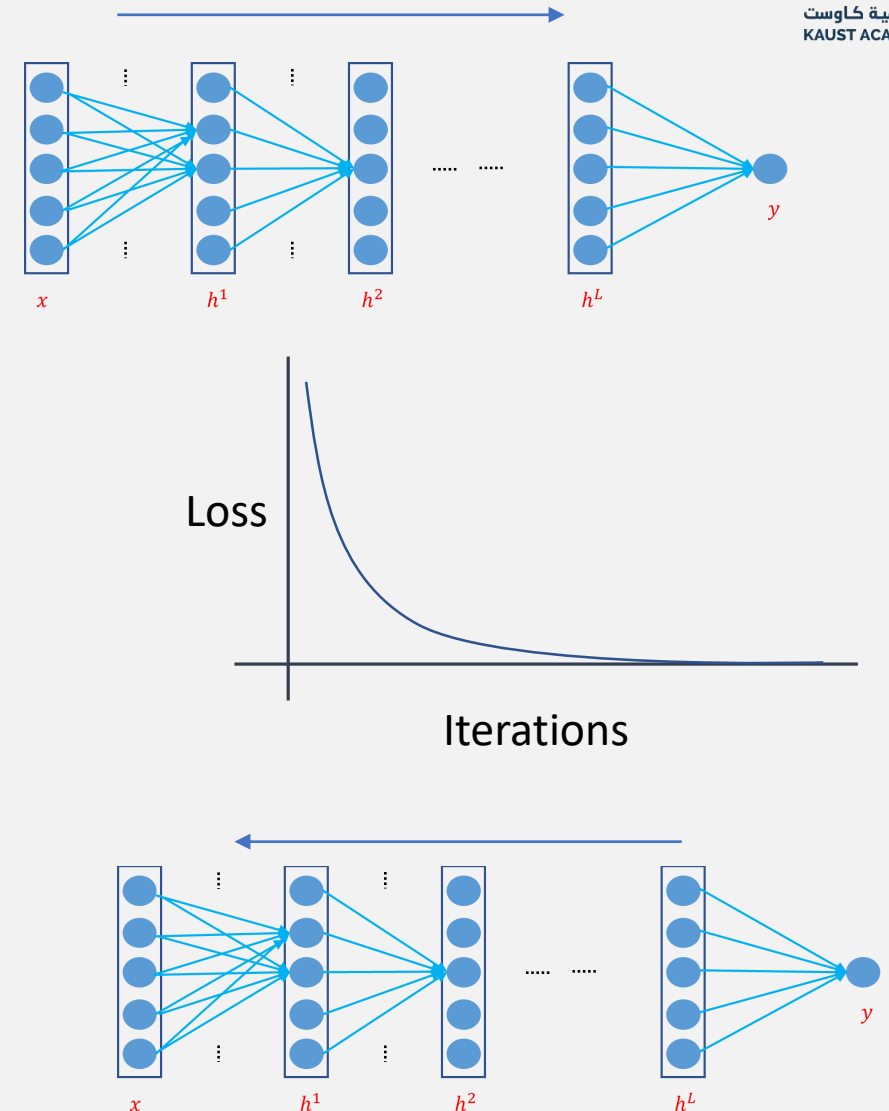
Backpropagation

- **Solution:** Backpropagation calculates the gradients of the loss with respect to each weight and bias.
- Backpropagation is a common method for training a neural network.
- Optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs

Backpropagation and Gradient Descent

Steps:

1. Do forwards propagation
2. Compute the Loss/Error. (For instance, MSE)
3. Backward Pass
 - Use the **chain rule** to compute gradients
 - Adjust weights and biases
4. Repeat forward and backward passes to reduce the loss over time.



Backpropagation and Gradient Descent

Regularization Techniques (Dropout
and Batch Normalization)

Backpropagation and Gradient Descen

What is the problem with Deep ANN?

- Consider an image classification problem (indoor/outdoor scene).
- Image size = $64 \times 64 \times 3 = 12288$ (input layer)
- Suppose two hidden layers $h1 = 1000$ neurons, $h2 = 500$ neurons
- Outputs = 2 neurons (one for each class)
- How many parameters would we need to train??
- $12288 \times 1000 + 1000 \times 500 + 500 \times 2 = 12789000$

(12.7 million) very small image and very small network



$64 \times 64 \times 3$



$64 \times 64 \times 3$

Backpropagation and Gradient Descen

What is the problem with Deep ANN?

- Consider an image classification problem (indoor/outdoor scene).
- Image size = $64 \times 64 \times 3 = 12288$ (input layer)
- Suppose two hidden layers $h1 = 1000$ neurons, $h2 = 500$ neurons
- Outputs = 2 neurons (one for each class)
- How many parameters would we need to train??
- $12288 \times 1000 + 1000 \times 500 + 500 \times 2 = 12789000$
(12.7 million) very small image and very small network



$64 \times 64 \times 3$



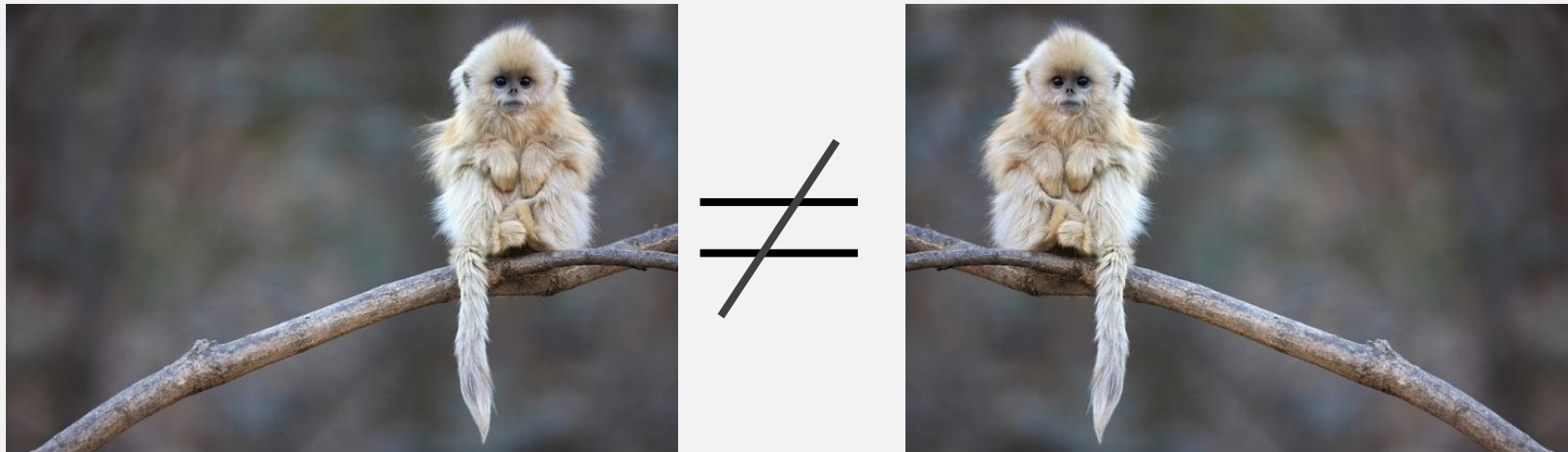
$64 \times 64 \times 3$

What if the image size = $256 \times 256 \times 3$, and network has 4 layers with 1000 neurons and 1000 outputs?

Backpropagation and Gradient Descen

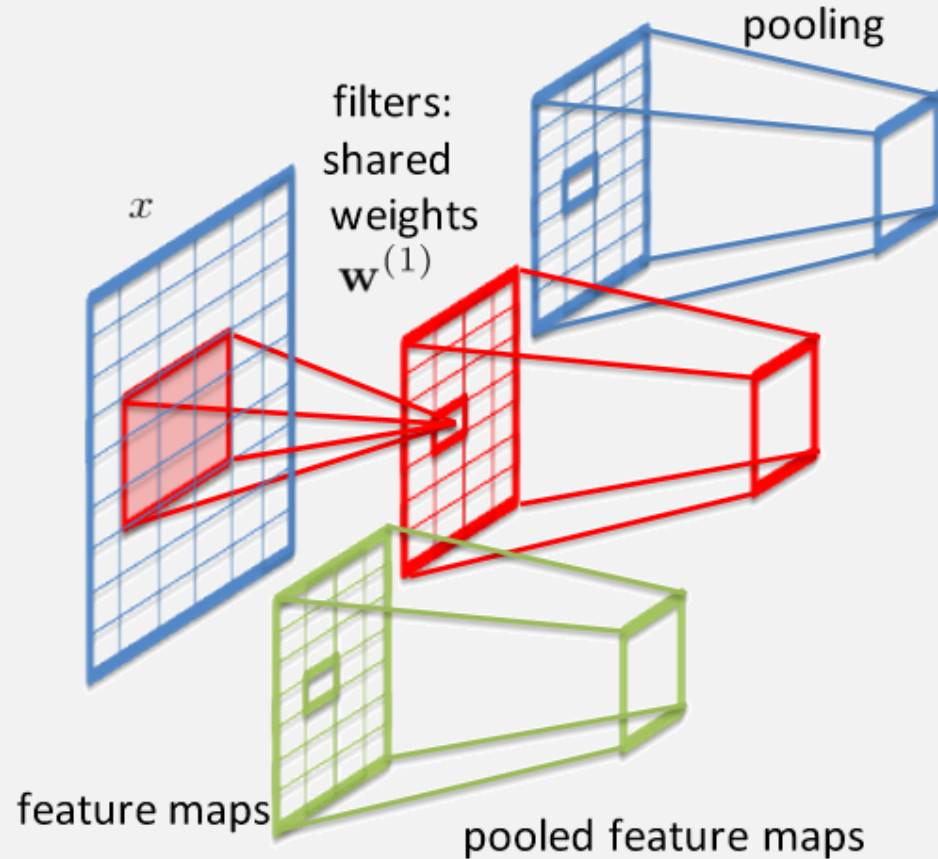
What is the problem with Deep ANN?

- Too Many Parameters:
- MLP[1000 1000 1000]-2000 000 parameters require too much training data.
- Computational and don't generalize well.
- No spatial invariance for images.

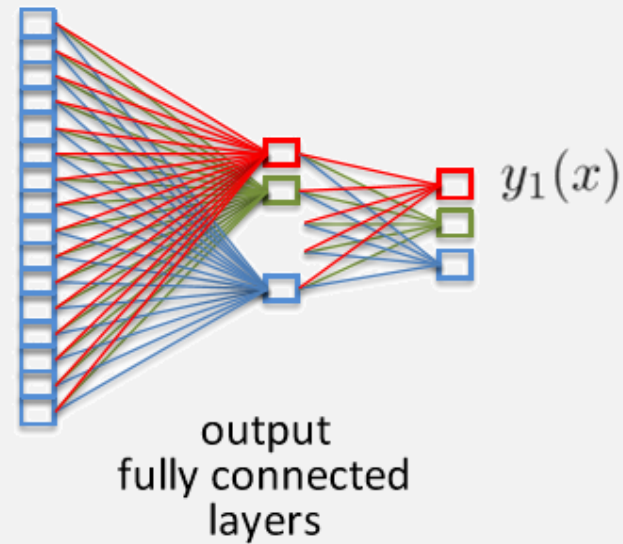


The Solution?

Convolutional neural networks (CNN)



3D Visualization of a Convolutional Neural Network



Brief about CNN



Yann LeCun, Professor of Computer Science
The Courant Institute of Mathematical Sciences,
New York University
Room 1220, 715 Broadway, New York, NY 10003, USA.
(212)998-3283

Email: yann@cs.nyu.edu

In 1995, [Yann LeCun](#) and [Yoshua Bengio](#) introduced the concept of convolutional neural networks

Brief about CNN

Convolutional Neural Networks are a special kind of multi-layer neural networks.

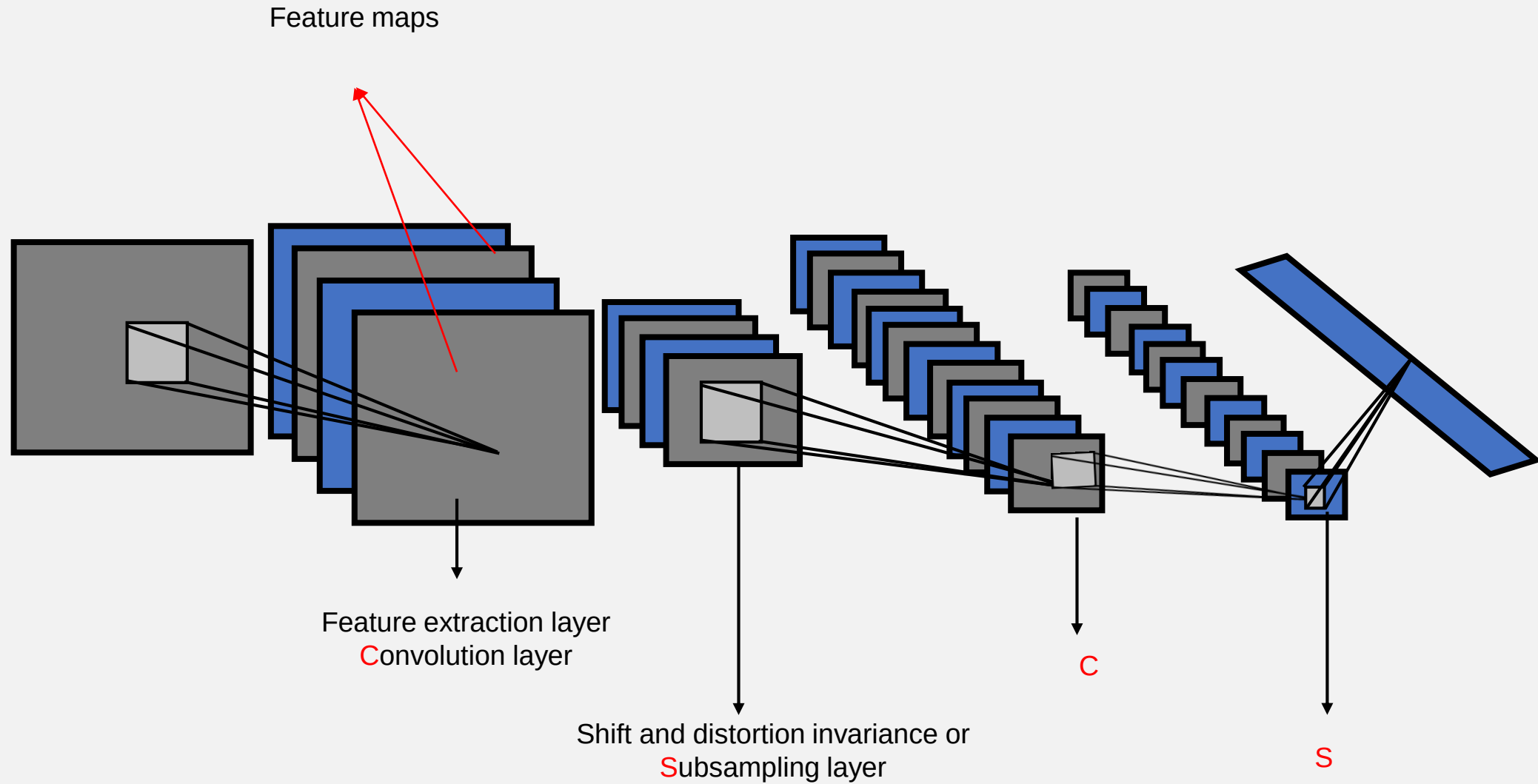
CNN is a feed-forward network that can extract topological properties from an image.

Like almost every other neural networks they are trained with a version of the back-propagation algorithm.

Convolutional Neural Networks are designed to recognize visual patterns directly from pixel images with minimal preprocessing.

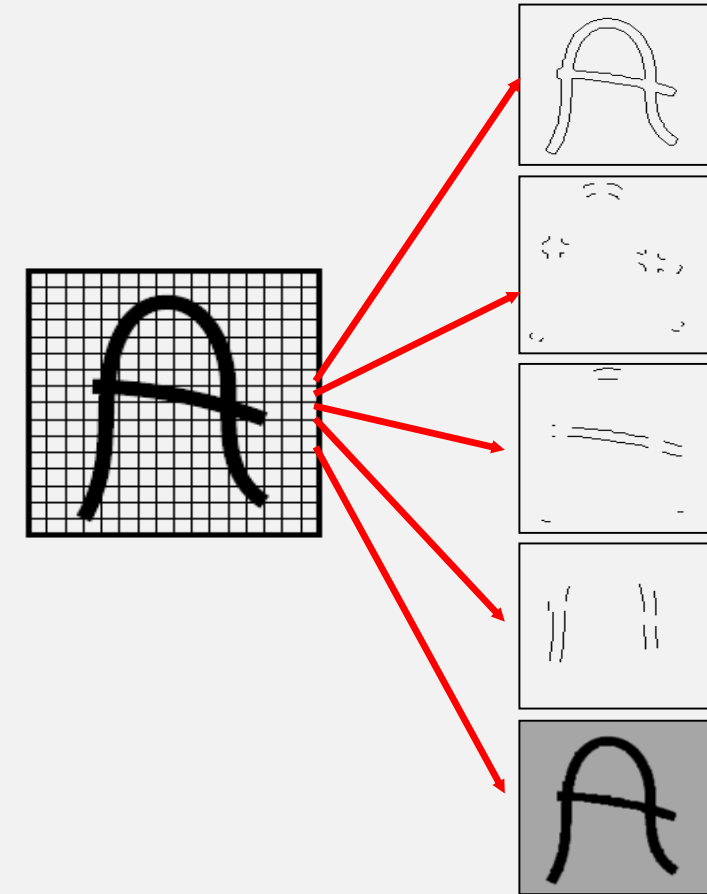
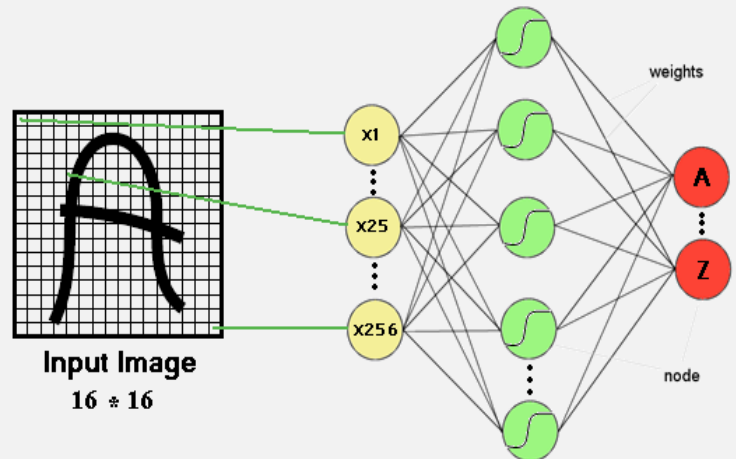
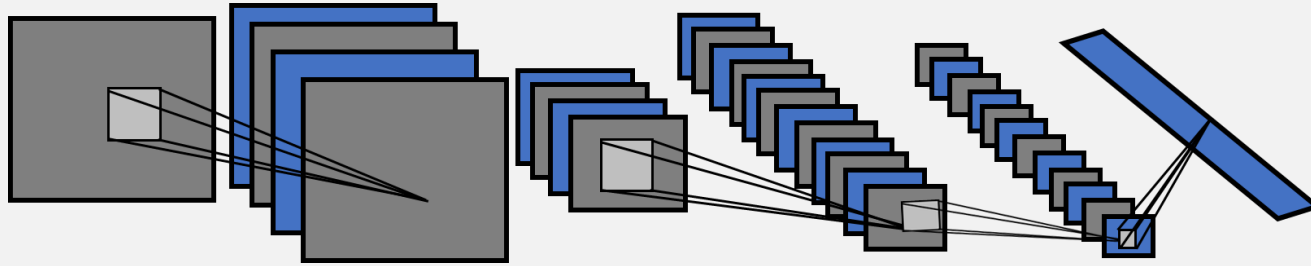
They can recognize patterns with extreme variability (such as handwritten characters).

CNN Building Blocks



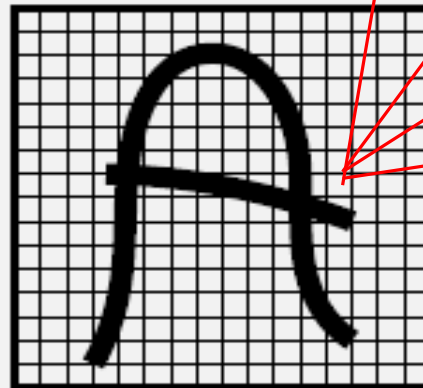
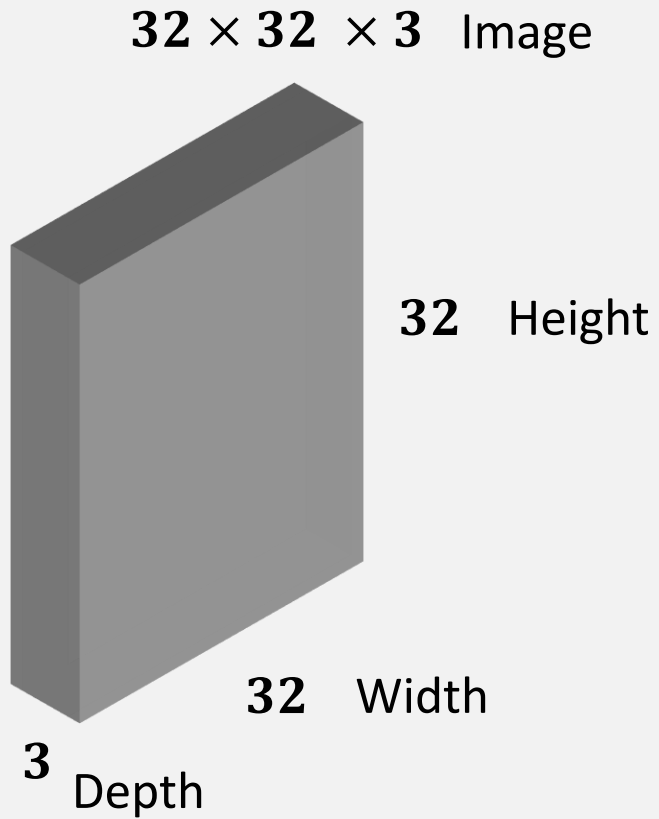
Convolutional Layer

Detect the same feature at different positions in the input image.



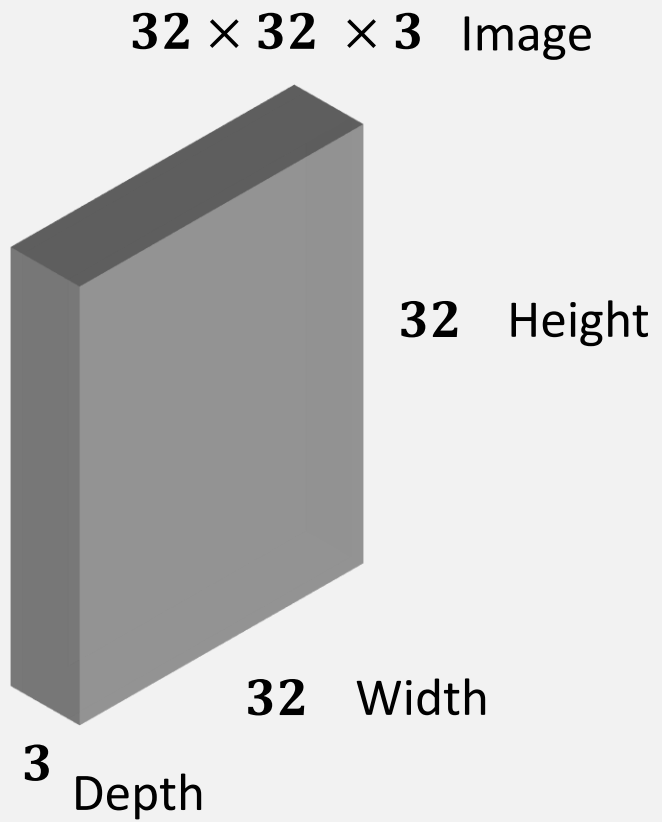
Features

Convolutional Layer

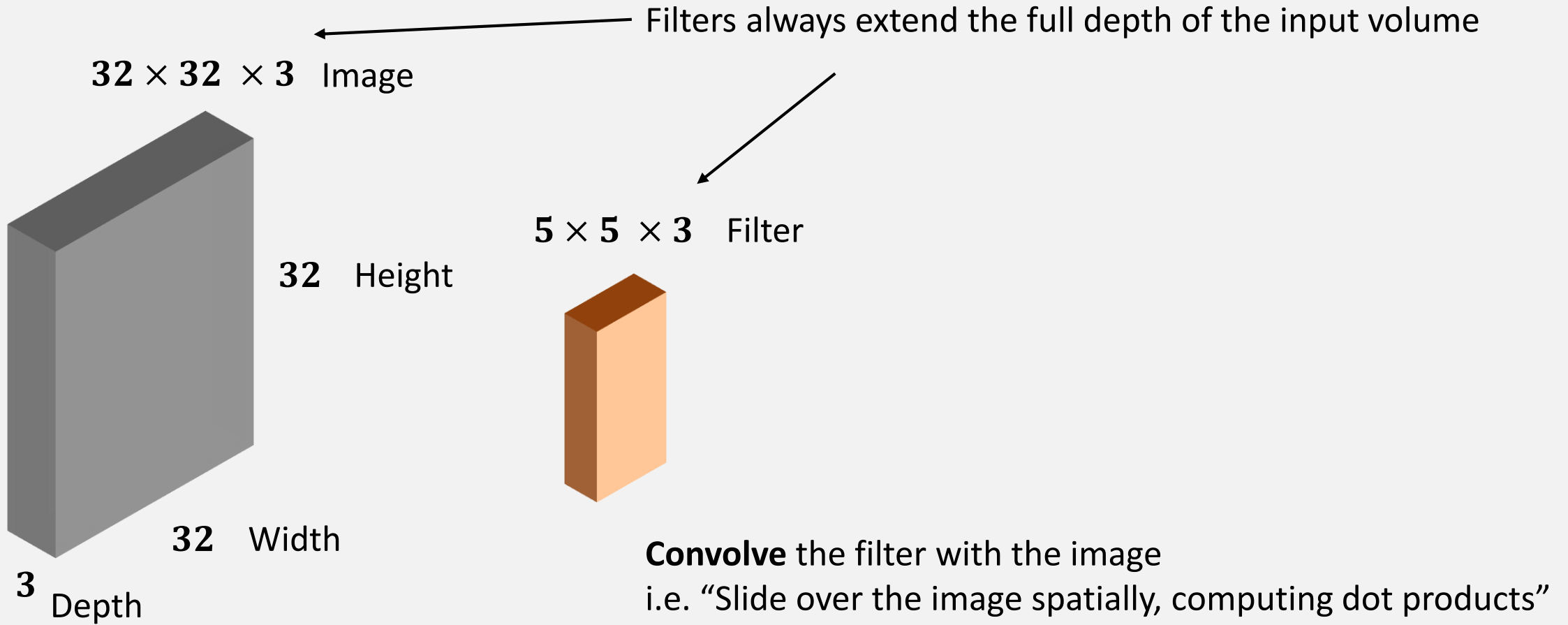


22	562	72	67	56	25	79	95	78	41
489	23	52	54	717	47	89	101	123	74
56	45	41	89	42	758	65	45	214	45
456	414	252	414	64	42	471	32	584	74
89	74	452	75	45	737	95	452	75	452
56	45	85	54	87	120	12	85	54	85
894	74	788	34	012	37	35	788	34	788
101	45	65	78	89	32	91	51	46	56

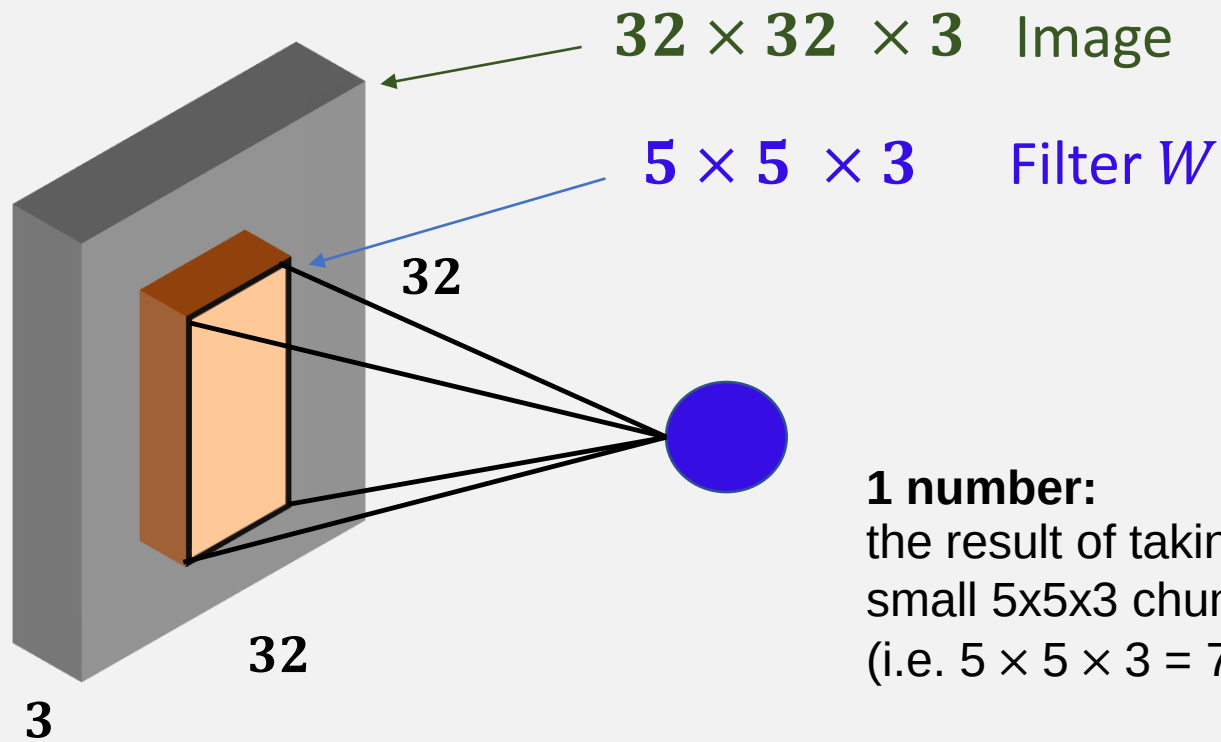
Convolutional Layer



Convolutional Layer



Convolutional Layer

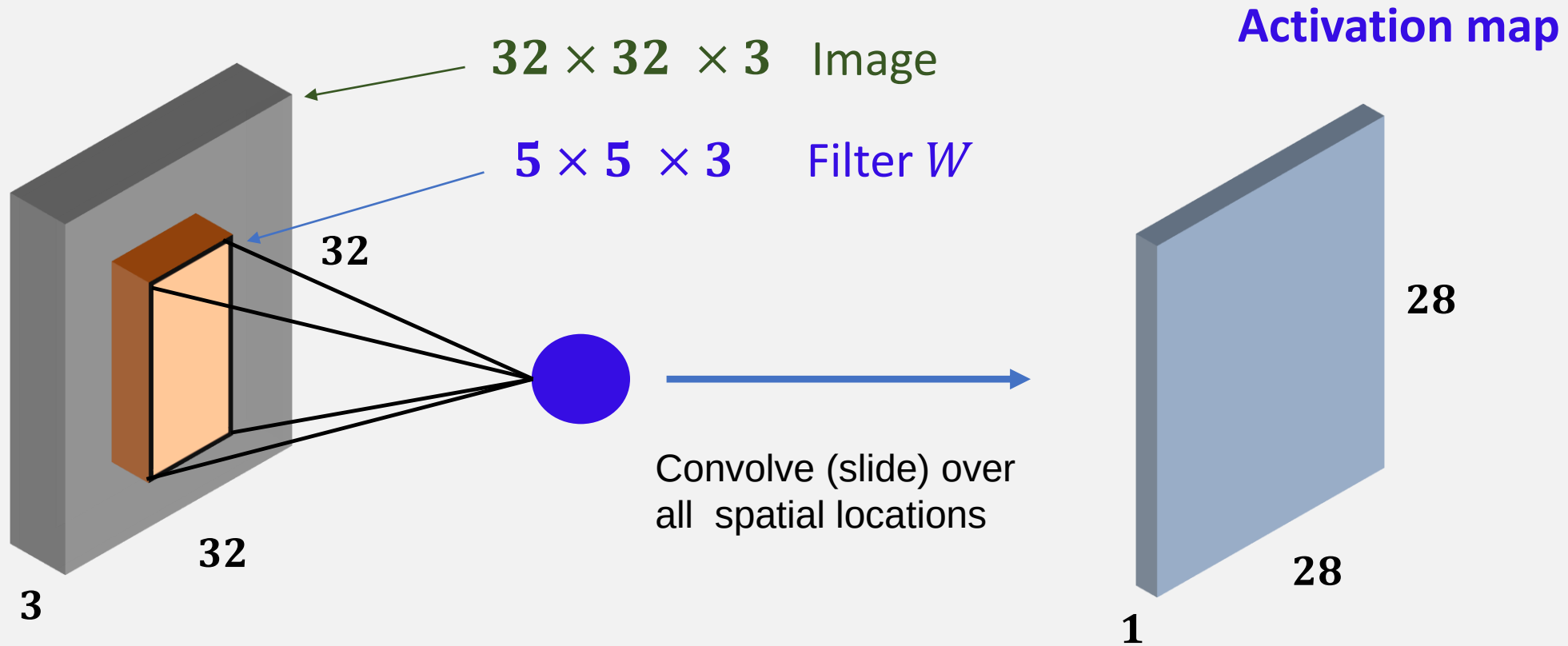


1 number:

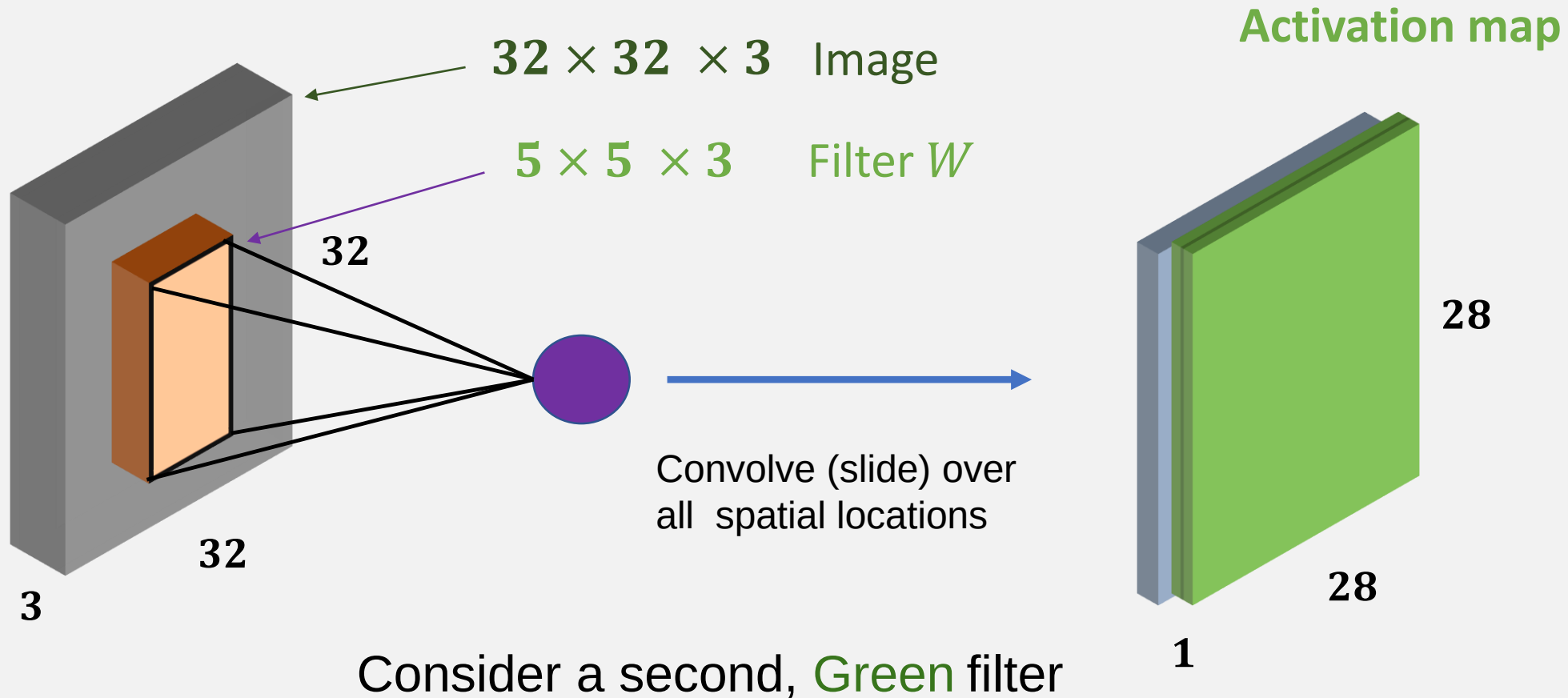
the result of taking a dot product between the filter and a small $5 \times 5 \times 3$ chunk of the image
(i.e. $5 \times 5 \times 3 = 75$ -dimensional dot product + bias)

$$w^T x + b$$

Convolutional Layer

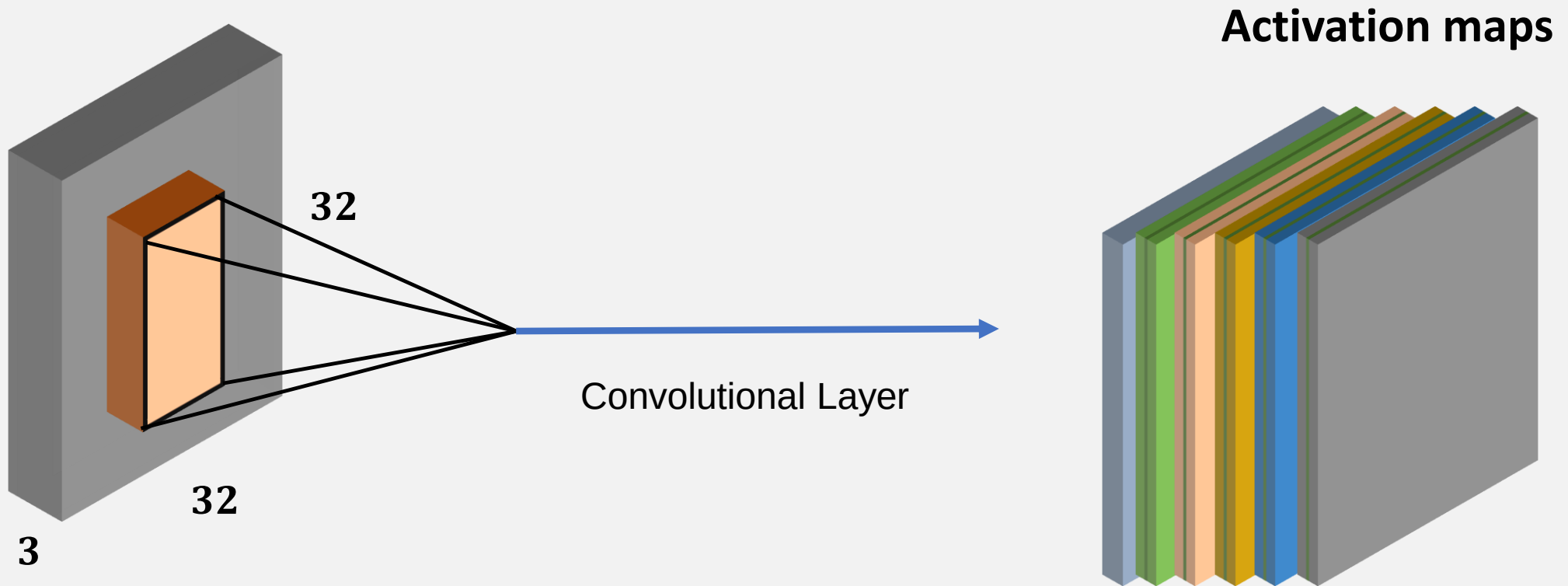


Convolutional Layer



Convolutional Layer

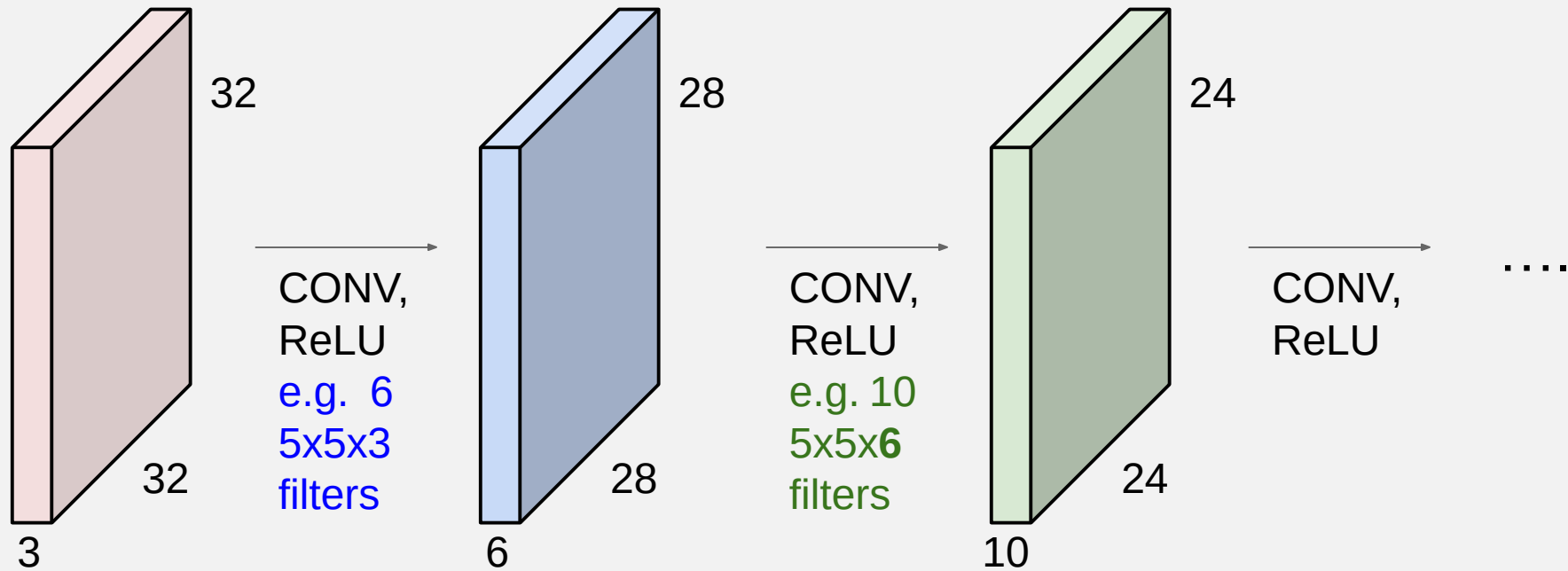
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps



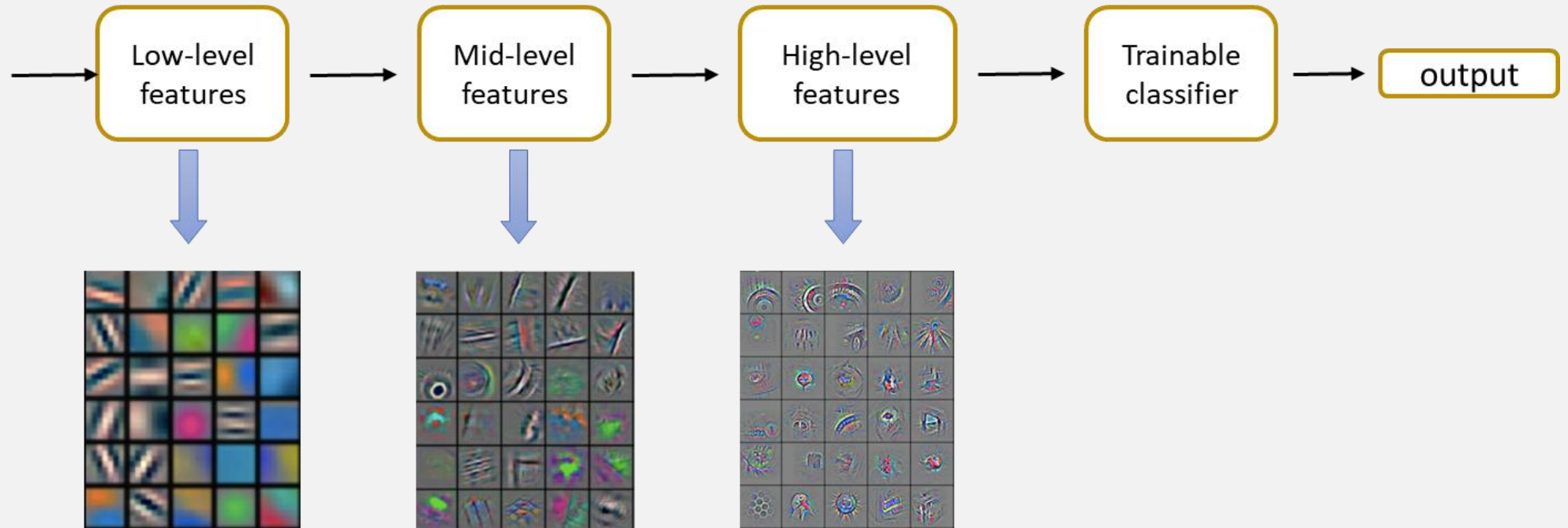
We stack these up to get 'new image' of size of $28 \times 28 \times 6$

Convolutional Layer

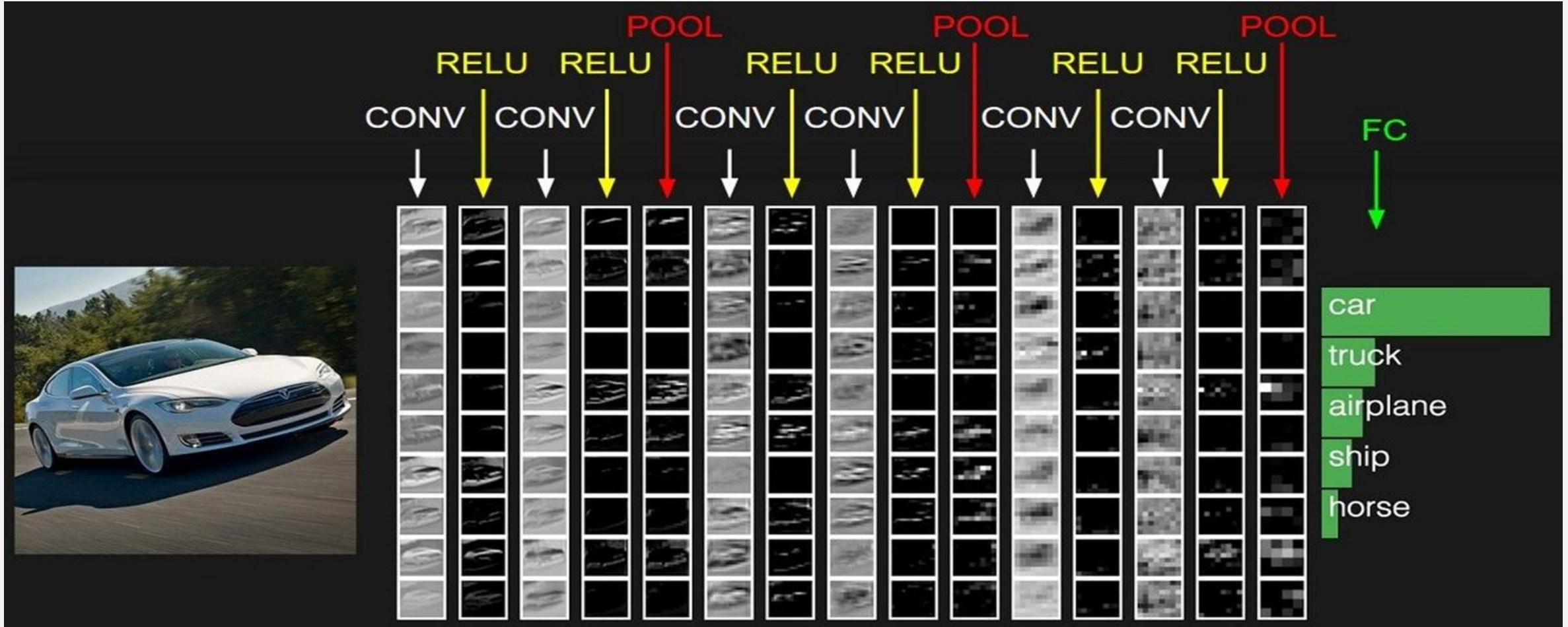
CNN/ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Convolutional Layer



Convolutional Layer



**More in Deep Learning such as CNN,
Stride in them, its development will
be covered in coming week.**